

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Zhang, Ashton, et al. *Dive into Deep Learning*. <https://d2l.ai/d2l-en.pdf>.

- **Required pages:** Chapter 3
- **Estimated reading time:** 75 minutes

2. Zhang, Ashton, et al. *Dive into Deep Learning*. <https://d2l.ai/d2l-en.pdf>.

- **Required pages:** Chapter 12, section 12.4 on Stochastic Gradient Descent
- **Estimated reading time:** 15 minutes

Module 7: Lesson 1

Introduction to Neural Networks



Outline

- ▶ Road map of the module
- ▶ Neural networks
- ▶ Linear regression as a neural network

Road map for the module

The final module of ML serves as an introduction to your next course on Deep Learning for Finance. Thus, we will start by revisiting some of the classic concepts (e.g., linear regression) using deep learning vocabulary. It will also help in introducing the environment we will be working with in the next course: Tensorflow.

- ▶ **Lesson 1** revisits linear regression as a neural network, introducing some important concepts that we will use later on (e.g., minibatch stochastic gradient descent, ...)
- ▶ **Lesson 2** introduces the Tensorflow environment on a practical linear regression setting: we will test if linear models have any market timing ability by evaluating smart beta trading strategies.
- ▶ **Lesson 3** introduces Multi-Layer Perceptron (MLP) networks.
- ▶ **Lesson 4** tackles the predictability problem of smart beta trading strategies using an MLP neural network to compare the performance of the algorithms with respect to the classic linear case.

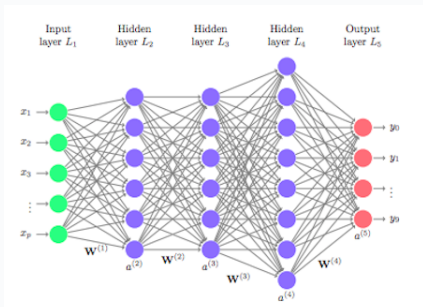
So, let's start by looking at linear regression, something that you are already very familiar with, through the lens of a neural network.

Neural Networks

The final module of ML serves as an introduction to your next course on Deep Learning for Finance. Thus, we will start by revisiting some of the classic concepts (e.g., linear regression) using neural network vocabulary.

- The theoretical development of this lesson is on **Chapter 3** from the book by [Zhang, Aston, et al. *Dive into Deep Learning*. 2022.](#)

A **neural network (NN)** can simply be seen as a series of different algorithms that aim to recognize patterns in the input data by processing information through different layers and neurons:



Linear Regression as a Neural Network

Before dealing with deep neural networks, it will be useful to think about a linear regression problem as a single-layer neural network.

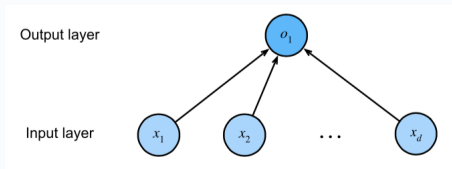
- We know already the classic linear regression equation form:

$$y = Xw + b$$

where w that minimize a squared error (loss) function can be expressed in closed-form as:

$$w^* = (X^T X)^{-1} X^T y$$

- Now, linear regression can be expressed as a single-layer neural network:



Let's see how this works.

Linear Regression as a Neural Network

There are 4 main features that will characterize a simple neural network model such as the one for linear regression:

1. The model
2. The loss function
3. The optimization algorithm
4. The training function

⇒ In constructing a model, we will have to make choices to define each of these features.

Most of the time, as we work with libraries like Tensorflow, defining all these features would be just a few lines of code. However, it is crucial that we understand what is going on behind the scenes because this will allow us to:

- ▶ Make better choices depending on the tasks at hand.
- ▶ Identify where problems in model performance stem from.

Next, we review these different features...

Linear Regression as a Neural Network

1. The model

In the case of linear regression, the model is quite simple, and it simply encompasses 1 layer of neurons (each corresponding to a different input) that are combined to form the output in the following linear way:

$$y = Xw + b$$

that is, going **forward** from the input layer to the output layer is done by a matrix-vector product of input features X and weights w and then adding the scalar b (bias term) using broadcasting.

→ Of course, we are aware that, in the linear case, there is a closed-form solution to the form of weights (w) that minimizes the squared error.

→ However, we are going to use neural network features relying on iterations to obtain the solution to the problem:

- ▶ How do we establish the initial values for the weights (w)?
 - Matrix/vector of zeros? → No good
 - Random numbers? → Yes, better! (how?)
- ▶ How do we establish initial values for bias term (b)? Zero!

Linear Regression as a Neural Network

2. The loss function

We need a loss function to use in our optimization problem with neural networks. This loss function will measure the fitness of the model (or better said, unfitness).

It is common, especially in the linear case, to use a squared error loss function, defined such that for each example i :

$$\ell^{(i)}(w, b) = \frac{1}{2} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

Note that the $1/2$ is just a convenient way to express it to cancel out the power in the derivative.

If we want to measure the fit of the whole model we simply average on n training set examples:

$$L(w, b) = \frac{1}{n} \sum_{i=1}^n \ell^{(i)}(w, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} \left(w^T x^{(i)} + b - y^{(i)} \right)^2$$

Linear Regression as a Neural Network

3. The optimization algorithm

This is one of the most important pieces to understand in the development of a model. First, it is important we understand the main optimization technique most algorithms rely on: **stochastic gradient descent (SGD)**

We already know how gradient descent works. **SGD** reduces computational costs because the gradient is computed from a uniform random sample of i examples in the data. Thus, we update $\mathbf{x}$, using a learning rate η as:

$$\mathbf{x} \leftarrow \mathbf{x} - \eta \nabla f_i(\mathbf{x})$$

Moreover, to increase speed we are going to use **minibatch SGD**:

- ▶ When updating the SGD algorithm, every observation takes time. Parsing the entire dataset for every update of the algorithm does as well.
- ▶ So, a faster intermediate strategy is taking a **minibatch** of observations, where weights are updated as:

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}_t} \partial_{\mathbf{w}} \ell^{(i)}(\mathbf{w}, \mathbf{b})$$

Linear Regression as a Neural Network

4. The training

Training is the moment of the DL algorithm where we put all the other parts together. As we have already mentioned, the training of a DL algorithm is aimed at minimizing the loss function with an iterative process:

- ▶ In each **iteration**, we take a minibatch of examples to compute gradients and update model parameters.
- ▶ In each **epoch**, we iterate through the entire training set.

This is essentially a loop. What is the structure of the training process?

1. Initialize parameters $\rightarrow (w, b)$

2. Loop **until done**:

- ▶ Compute gradients: $g \leftarrow \partial_{(w,b)} \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \ell(x^{(i)}, y^{(i)}, w, b)$
- ▶ Update parameters: $(w, b) \leftarrow (w, b) - \eta g$

$\Rightarrow$ **What does "until done" mean?** Stop when it reaches a pre-specified condition (e.g., max epochs, min error, etc.)

Summary of Lesson 1

In Lesson 1, we have looked at:

- ▶ Basics of Neural Networks (NNs)
- ▶ Linear regression as an NN

⇒ **References for this lesson:**

Chapters 3 and 12.4 in [Zhang, Aston, et al. *Dive into Deep Learning*. 2022.](#)

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement linear regression as a neural network using Tensorflow in a finance context application.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Zhang, Ashton, et al. *Dive into Deep Learning*. <https://d2l.ai/d2l-en.pdf>.

- **Required pages:** Chapter 12, section 12.10 on ADAM optimizer
- **Estimated reading time:** 10 minutes

Module 7: Lesson 2

Momentum Timing with Linear Regression as a NN



Outline

- ▶ Smart beta / factor investing
- ▶ Linear regression as a neural network for factor timing
- ▶ Backtesting the 'Mom timing' strategy

Factor investing

In this lesson, which is focused on working with the accompanying Jupyter Notebook, we will implement a linear regression model using neural networks in the Tensorflow-Keras framework.

- ▶ **What are we going to use our neural network for?**

We will build a predictive model to develop a trading strategy based on factor investing (or smart beta).

- ▶ **Factor investing?**

- **What is a factor?**

- Essentially, a characteristic that systematically explains the cross-section of stock returns (e.g., size, value, momentum, etc.).

- **What is factor investing?**

- These factors help explain the cross-section of stock returns, so we can invest following them.

- Value example: a portfolio that goes long on 10% firms with the lowest book-to-market (B/M) and short on the 10% with the highest B/M delivers an above-market return (factor premium).

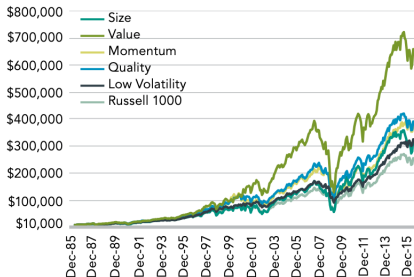
- Factor investing uses a systematic approach to try and capture all these factor premiums

Factor Investing

But all these factor premiums are time-varying and a specific factor does not always outperform.

EXHIBIT 7: Growth of \$10,000 Investing in Factor Portfolios vs. the Broader Market (1985–2015)

These five factors have outperformed over time

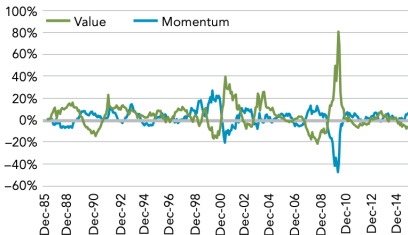


Returns are cumulative and assume reinvestment of dividends. Past performance is no guarantee of future results. Factor definitions consistent with those shown in previous exhibits. Source: FactSet, as of Mar. 31, 2016.

EXHIBIT 8: Value and Momentum Cyclicalities

Most factors are not highly correlated, so diversifying among them may improve risk-adjusted returns over time

Rolling Annual Excess Return



Value represented by the equal-weighted top quintile (by book-to-price ratio) of the Russell 1000 Index. Momentum represented by the equal-weighted top quintile (by trailing 12-month returns) of the Russell 1000 Index. Source: FactSet, as of Mar. 31, 2016.

Our factor investing strategy

All these different factors offer a source of premium that is time-varying. In this lesson's notebook, we will implement a factor-timing strategy for the **momentum** factor.

- ▶ What is **momentum**?

Refers to the documented fact that stocks that perform the best (worst) over a three- to 12-month period tend to continue to perform well (poorly) over the subsequent three to 12 months.

- ▶ What does **momentum timing** require?

Simply, that we can predict the returns that the momentum factor is going to offer in the future.

In our context, successfully timing momentum requires that our NN model is able to predict momentum returns relatively well using past momentum returns as inputs.

- ▶ What **process** are we going to follow?

1. Predictive model → Linear regression neural network
2. Develop trading strategy based on momentum timing
3. Backtesting the momentum timing trading strategy to evaluate its performance

Neural network for momentum timing

Our first step is to construct a neural network model that aims to predict momentum returns. To do so, we will gather data on daily historical momentum factor returns. Our model will then comprise:

- ▶ **Output:** Momentum factor returns for the next 60 days.
- ▶ **Inputs:** Momentum-sorted portfolio returns for:
 - 5 different time-spans: 10, 25, 60, 120, and 240 previous days.
 - 3 different portfolios: the top 10% and bottom 10% of firms by previous momentum, and a Mom portfolio that buys the top and sells the bottom decile portfolios.

This means a total of $3 \times 5 = 15$ inputs. Thus, the model will train 16 parameters: 15 weights (w), plus the bias term (b).

- Before training the model, we select 60% of observations (careful about the chronological order!) for training and the remaining 40% for test.
- We also need to perform some scaling of the variables (inputs and outputs) following the classic MinMaxScaler in sklearn.

Neural network for momentum timing

- How do we train the model? → Using a single-layer NN!

```
import tensorflow as tf

model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Dense(1))

hp_lr = 1e-5

adam = tf.keras.optimizers.Adam(lr=hp_lr)
model.compile(optimizer = adam, loss= 'mean_squared_error')
model.fit(X_train, y_train, epochs=50, batch_size=32, verbose=2)
```

- Which optimizer are we using? → Adam

- Adam is one of the most common deep learning optimizers based on SGD.
- Adam uses leaky averaging in the gradient descent process, taking into account both the first and second moments of the gradient.
- For more on Adam, see Section 12.10 in the book by [Zhang et al.: Dive into Deep Learning](#).

Model performance and momentum timing

Once we have trained our model in sample, it is time to assess its predictive ability and whether we can use it for trading purposes.

► Assessing model fit

- We already know that the R^2 is a classic measure in linear regression to evaluate goodness of fit in a model. The in-sample R^2 we are used to is, however, of no use in here. Remember that we aimed at constructing a model that can predict out-of-sample momentum returns to develop a trading strategy.
- Hence, we will rely on Campbell and Thompson (2008) to develop an out-of-sample R^2 that captures models' predictive power in future stock returns:

$$R_{OS}^2 = 1 - \frac{\sum_{t=1}^T (r_t - \hat{r}_t)^2}{\sum_{t=1}^T (r_t - \bar{r}_t)^2}$$

- A high R_{OS}^2 is a good indicator that our model will be able to help us in timing the momentum factor.

However, note that even a poor R_{OS}^2 can have some use in practice because even if the model fails a lot at predicting outcomes, the incorrect guesses may imply small losses, while the correct guesses may convey large wins!

Model performance and momentum timing

A common way to evaluate the viability of a trading strategy (not the model's predictive power!) is to perform a **Backtest**.

► Backtesting trading strategy

- Like traveling back in time and assessing how well/bad would we have done if we followed the strategy.
- It is extremely important that we **do not cheat** when performing backtesting. That is, among other things, we can only consider information that was known at each point in time when we make the trading decisions.
- If we deployed a trading strategy that buys (sells) the momentum portfolio when our model predicts a positive (negative) return the next day, this is how we would have done:



Summary of Lesson 1

In Lesson 2, we learned how to:

- ▶ Work with neural networks (NNs) in Tensorflow Keras
- ▶ Implement a linear regression problem as an NN
- ▶ Develop a smart beta trading strategy based on the predictions of a linear regression NN model for the momentum factor
- ▶ Perform backtesting of a trading strategy

⇒ **References for this lesson:**

Chapters 12.10 on [Zhang, Aston, et al. *Dive into Deep Learning*. 2021.](#)

TO DO NEXT: Go to this lesson's associated Jupyter Notebook. There, you will work on a linear regression model using a neural network to develop a trading strategy that aims to time the momentum factor.

In the next lesson, we will introduce a much more complex NN, multilayer perceptron (MLP).

MACHINE LEARNING

MODULE 8 | LESSON 2

Linear Models in TensorFlow: Timing Factors and Smart-Beta Strategies

Reading Time	90 minutes
Prior Knowledge	Linear Algebra, Python
Keywords	Linear Regression, Classification, Machine learning

In this lesson, we will take our first look at a neural network (NN) algorithm in the simple framework of linear regression. As you will see, there are lots of different applications of NNs. We are going to tackle a finance one. More specifically, we will try to use this deep learning (not so deep, yet) tool to predict the performance of smart-beta strategies. In this lesson, we will tackle this problem using linear regression (a single-layer, fully-connected NN). In lesson 3 of this module, we will repeat this same exercise using multilayer perceptrons (MLP), a real (but simple) deep network. Then, we will evaluate the benefits of more complex networks with several hidden layers.

But before going there, what are smart-beta strategies?

1. Smart-Beta Strategies (Factor Investing)

Smart-beta strategies are basically those based on the different market anomalies known as **risk factors**. Hence, sometimes, you will see these also referred to as **factor investing**.

- **What is a factor?**

A factor is simply a characteristic (e.g., macroeconomic, style, etc.) that explains the cross-section of stock returns. The academic literature has, for a long time now, investigated the characteristics that defy the market efficient view. In the process, researchers have come up with a good number of factors (or market anomalies) that help explain the cross-section of stock returns in addition to the exposure to market return (CAPM β).

(If you are not familiar with the CAPM model, you can check a very intuitive (and old) article on HBR here: <https://hbr.org/1982/01/does-the-capital-asset-pricing-model-work>)

So these factors help explain the cross-section of stock returns, but how exactly? Let's illustrate this with the example of the **value** factor, one that you all have probably heard of at some point.

Value is tightly linked to a firms' **book-to-market (B/M)** ratio, which is simply the market price of a stock over its book value. This acts as a direct indicator of the relative prospects of the firm. In probably one of the most famous finance papers of all time, Fama and French determined that firms with a lower B/M tend to be strong performers (on average), while firms with a high B/M are usually weak. Importantly, this out- and under- performance is not explained by other known characteristics such as exposure to market risk or firm size.

Here, you can have a look at some early papers by Fama and French on the subject.

- Fama, Eugene F., and Kenneth R. French. "The Cross-Section of Expected Stock Returns." *The Journal of Finance*, vol. 47, no. 2, 1992, pp. 427-465, <https://www.jstor.org/stable/2329112>.
- Fama, Eugene F., and Kenneth R. French. "Common Risk Factors in the Returns on Stocks and Bonds." *Journal of Financial Economics*, vol. 33, no. 1, 1993, pp. 3-56, ([https://doi.org/10.1016/0304-405X\(93\)90023-5](https://doi.org/10.1016/0304-405X(93)90023-5))

So, the bottom line is that according to these studies, we should be able to make money, on average, by simply going long on a portfolio of firms with very low B/M and/or going short on a portfolio of firms with high B/M. The question now is why?

There are various explanations for this phenomenon. Probably the most accepted one, and the one defended by Fama and French in all their subsequent papers, is that B/M proxies for a **common risk factor** in the cross-section of stock returns. That is, the kind of strategy we described yields a profit because it also implies assuming more risk. For example, since high market prices relative to book values (that is, low B/M) are probably driven by better future prospects of the firm, there is an underlying risk of realization of these prospects.

Hence, B/M constitutes a factor that helps explain the cross-section of stock returns, but also a characteristic that would aid in selecting our investments. We could then construct a **B/M factor (or value factor)** that goes long in a portfolio of the 10% of firms with the lowest B/M, and short-sells a portfolio of the 10% of firms with the highest B/M. Such a portfolio would earn a **factor premium**. Since there are quite a few factors identified by the academic literature (some more widely accepted as such than others), there are a bunch of different characteristics that we can use for our investments.

(Here, you can find a recent paper by Harvey et al. (2016) exploring the "zoo" of new factors found in the academic literature. For some of them, the evidence supporting their condition of "common risk factor" is only weak, but some are unambiguously classified as such:

<https://academic.oup.com/rfs/article/29/1/5/1843824?login=false>)

- **Factor Investing**

The kind of investment strategy that relies on these factors is usually referred to as **factor investing** or **smart beta** strategies. Indeed, the last few years have witnessed a wild explosion in the use of these strategies. Since the trading decision (e.g., buy low B/M firms, short-sell high B/M firms) of these strategies is very systematic and has a clear rule, many important investment managers like Vanguard or BlackRock have specifically designed products using these notions. Most often, these investment managers create products like ETFs (exchange-traded funds) that track one (or several) of these factors. But it is not only widely known passive investors such as Vanguard or BlackRock that rely on factor investing as a product to market to clients. Prominent players do also undertake this systematic approach for their own gain, such as AQR, a hedge fund. Next, you can see each of these players' views on factor investing:

- [Vanguard's factor-based strategies](#)
- [BlackRock's factor-based investing](#)
- [AQR's views on factor investing](#)
- [Fidelity's overview of factor investing](#)

- **Momentum Factor**

In this notebook, we are going to focus on one of the most profitable and persistent factors over time: **momentum**. Individual stocks' momentum is, in its simplest form, a concept similar to *inertia*. It refers to the documented fact that stocks that perform the best (worst) over a three- to 12-month period tend to continue to perform well (poorly) over the subsequent three to 12 months:

- *Here, you have the first paper that formally documented the returns to a momentum strategy by Jegadeesh and Titman:*

<https://www.jstor.org/stable/2328882>

Momentum is one of the most widely used factor strategies, as it is easy to implement, requires a lower amount of input data, and offers great returns over time. In the following paper, Asness et al. document a prevalence of value and momentum factors in many asset classes:

- Asness, Clifford S., et al. "Value and Momentum Everywhere." *The Journal of Finance*, vol. 68, no. 3, 2013, pp. 929-985,
[https://onlinelibrary.wiley.com/doi/abs/10.1111/jofi.12021?](https://onlinelibrary.wiley.com/doi/abs/10.1111/jofi.12021?casa_token=vxQDrj_UikoAAAAA:hXo8tu-)
[casa_token=vxQDrj_UikoAAAAA:hXo8tu-](https://onlinelibrary.wiley.com/doi/abs/10.1111/jofi.12021?casa_token=vxQDrj_UikoAAAAA:hXo8tu-)

- **Factor Timing**

At this point, we know that investing in these so-called factors seems a reliable source of returns in the long run. But, of course, the performance of these different factors is not always the same across time. Some factors perform better than others under certain circumstances and vice versa. What will be more interesting to us is being able to perform some **factor timing**. That is, similar to market timing, we can go long in the factor when we predict that a specific characteristic is going to do especially well and go short on it when we predict it is going to do badly.

Despite this effective strategy, unfortunately, factor timing is not so easy to do in practice. Being able to time the factor would mean that past returns of the factor serve well in explaining the future performance of the factor. The prediction of past factor returns is actually a very challenging task. Put it in Cliff Asness's (Founder & Managing Principal at AQR) words: "**Factor timing is likely even harder than market timing.**"

There are, still, some papers in the academic literature that tackle the question of factor timing, specially for momentum, with mixed results. This is actually dense literature, but we offer you some examples of papers exploring this avenue:

- Daniel, Kent, and Tobias J. Moskowitz. "Momentum Crashes." *Journal of Financial Economics*, vol. 122, no. 2, 2016, pp. 221-247, <https://www.sciencedirect.com/science/article/pii/S0304405X16301490>.
- Ehsani, Sina, and Juhani T. Linnainmaa. "Factor Momentum and the Momentum Factor." *The Journal of Finance*, vol. 77, no. 3, 2022, pp. 1877-1919, https://onlinelibrary.wiley.com/doi/full/10.1111/jofi.13131?casa_token=oWY7-8sCYqkAAAAA%3A-HrzsyeMLuXf0lQyYMT9vxFNyNQugJsMfaY39EAeTCjCIDIAPHdU_fqS4z2-LpVrUO7li2wQmlphYXA.
- Moskowitz, Tobias J., et al. "Time Series Momentum." *Journal of Financial Economics*, vol. 104, no. 2, 2012, pp. 228-250, <https://www.sciencedirect.com/science/article/pii/S0304405X11002613>.
- Huang, Dashan, et al. "Time Series Momentum: Is It There?" *Journal of Financial Economics*, vol. 135, no. 3, 2020, pp. 774-794, <https://core.ac.uk/download/pdf/287750773.pdf>.

- **What are we going to do in this notebook then?**

In this notebook, we will build a neural network, under a linear model, that aims to predict the future returns of the momentum factor. If we are successful in our

endeavor, this could grant us with a very valuable investment strategy to allocate our money!

As you can probably guess at this point, a linear model is not going to offer a decent prediction of future momentum returns. However, this is as good excuse as any to learn how to build a simple deep learning model. In fact, in the last lesson of the module we will revisit this problem under a more complex (although still not too complex) neural network. This way, we will be able to observe the power of deep learning in these kinds of finance prediction problems.

2. Data Sources

First and foremost, in order to build a deep learning model, we need data. In this case, we are going to take as inputs the returns from a momentum factor. Using these inputs, we will then aim to predict next period momentum factor returns using as inputs past returns. If we succeed, this means there is a way to time the momentum factor so that a strategy that goes long or short in the factor when the prediction indicates so should yield a better return than a simple buy-and-hold strategy. First, we will use deep learning to build our prediction model, then we will see if the resulting strategy does outperform.

- **Which input data and where do we obtain it?**

As mentioned before, the data for evaluating the timing of the momentum factor is actually somewhat easy to obtain because it only requires past returns of the momentum factor.

Remember that constructing a momentum factor implies going long on a portfolio of top recent winners (say, the top 10% of firms that did best in the recent past) and short on a portfolio of top recent losers (say, the top 10% that did worst in the recent past). There are many ways (i.e., time-horizons) to construct the momentum factor portfolio returns. Here, we will stick to the classic definition and compute the momentum factor using the returns from the prior (-2, -12) months. That is, if we were to invest in the momentum factor portfolio today at time t , we will take the returns of our universe of firms from $t - 12$ months to $t - 2$ months. The top 10% of firms that did best in that period will be our recent winners and, thus, we will be going long on them. Conversely, the 10% of firms with the poorest returns in the period will be our recent losers and we will be going short on those stocks. The return of the momentum factor tomorrow will be the return of such a portfolio from t to $t + 1$ day.

While constructing such data is a somewhat easy task, it will get easier for us. Here, we will rely on Prof. Ken French's data library, which contains freely available data on a lot of different portfolios sorted by characteristics (e.g., factors). This is a vast and handy database to have in mind whenever you are undertaking these kind of tasks:

- Prof. Ken French's Data Library:
https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html

Now, interestingly for us, one of the pieces of data in Prof. French's library is the daily return of the momentum portfolio ($t - 2, t - 12$), together with the returns of the 10 portfolios resulting from sorting stocks into deciles according to their past performance. More specifically, this will be the data we will be using:

- Daily returns of 10 Portfolios Formed Daily on Momentum:
https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/Data_Library/det_10_port_form_pr_12_2_daily.html

So, now that we know where to find the data for our deep learning model, let's begin building it!

3. Timing Momentum with (Linear Regression) Neural Networks

Let's start by visualizing the type of regression problem that we are going to encounter here: linear regression. This is the type of model that we will face:

$$y = Xw + b$$

Now, we need to make a few decisions in order to move forward. We will need to decide on the number of inputs and what inputs are we going to consider. Similarly, we need to decide which future return of the momentum factor we are going to try to predict. Before making those decisions, however, it may be informative to look at the actual data:

```
In [1]: import numpy as np
import pandas as pd
```

We will upload the file that contains the daily returns of 10 portfolios sorted on their past (-2, -12) months' returns:

```
In [2]: route = "10_Portfolios_Prior_12_2_Daily.csv"
```

```
In [3]: # Read the csv file again with skipped rows
df = pd.read_csv(route, index_col=0)
# Format the date index
df.index = pd.to_datetime(df.index, format="%Y%m%d")
# Build the MOM strategy: Long "Hi PRIOR" and Short "Lo PRIOR"
df["Mom"] = df["Hi PRIOR"] - df["Lo PRIOR"]
df.head()
```

Out[3]:

	Lo PRI OR	PRI OR 2	PRI OR 3	PRI OR 4	PRI OR 5	PRI OR 6	PRI OR 7	PRI OR 8	PRI OR 9	Hi PRI OR	Mo m
192 6- 11- 03	-0.12	0.60	-0.09	0.30	-0.51	-0.22	-0.12	0.50	0.13	1.28	1.40
192 6- 11- 04	0.65	1.82	1.34	0.61	1.01	0.64	0.82	0.44	0.48	0.40	-0.25
192 6- 11- 05	-0.84	-0.77	-0.22	-0.15	-0.02	-0.02	-0.07	0.36	0.20	0.08	0.92
192 6- 11- 06	1.03	0.28	0.24	0.40	0.19	0.64	0.10	0.10	0.39	-0.68	-1.71
192 6- 11- 08	-0.06	0.11	1.78	0.28	0.36	0.23	0.30	1.17	0.58	-0.18	-0.12

Now, as you can see, we have daily data on the return of each of these portfolios, courtesy of Prof. French, from November 3, 1926. Also, note that we are constructing the returns of the **momentum factor (portfolio)** simply as the returns from buying the stocks in the top decile ('Hi PRIOR'), and short-selling the stocks in the bottom decile ('Lo PRIOR'). Thus, for, say, November 3, 1926, the return of the '**Mom**' portfolio are: $1.28 - (-0.12) = 1.40\%$

Let's next discuss the different inputs and outputs we are going to consider:

3.1 Inputs and Outputs

Now, at this point in our task, it is clear that our aim is in predicting the future return of the Mom portfolio. It is not ex-ante clear which time-horizon of the Mom portfolio returns we should consider for our predictions. In principle, it does not make too much sense to predict a very short time-span nor a very long one. **For this example, let's try to predict the return of the next 60 days.** This will be our **output** variable. Then, you can play around with the code to check other time-spans.

The case of **inputs** requires more investigation. Once again, it is not ex-ante clear, but there are several papers in the literature that have already done some of the job for us. Specifically, the academic literature (see below) finds that the returns from the past: 10, 25, 60, 120, and 240 days are relevant in assessing future returns of the Momentum factor.

We thus calculate returns on these different horizons for the Mom portfolio, but also for the 'Hi' and 'Lo' ones, and use all as inputs:

```

In [4]: df["Ret"] = df["Mom"]
df["Ret10_MOMi"] = df["Mom"].rolling(10).apply(lambda x: np.prod(1 + x /
100) - 1)
df["Ret25_MOMi"] = df["Mom"].rolling(25).apply(lambda x: np.prod(1 + x /
100) - 1)
df["Ret60_MOMi"] = df["Mom"].rolling(60).apply(lambda x: np.prod(1 + x /
100) - 1)
df["Ret120_MOMi"] = df["Mom"].rolling(120).apply(lambda x: np.prod(1 + x /
100) - 1)
df["Ret240_MOMi"] = df["Mom"].rolling(240).apply(lambda x: np.prod(1 + x /
100) - 1)

df["Ret10_hi"] = df["Hi PRIOR"].rolling(10).apply(lambda x: np.prod(1 + x
/ 100) - 1)
df["Ret25_hi"] = df["Hi PRIOR"].rolling(25).apply(lambda x: np.prod(1 + x
/ 100) - 1)
df["Ret60_hi"] = df["Hi PRIOR"].rolling(60).apply(lambda x: np.prod(1 + x
/ 100) - 1)
df["Ret120_hi"] = df["Hi PRIOR"].rolling(120).apply(lambda x: np.prod(1 +
x / 100) - 1)
df["Ret240_hi"] = df["Hi PRIOR"].rolling(240).apply(lambda x: np.prod(1 +
x / 100) - 1)

df["Ret10_Low"] = df["Lo PRIOR"].rolling(10).apply(lambda x: np.prod(1 + x
/ 100) - 1)
df["Ret25_Low"] = df["Lo PRIOR"].rolling(25).apply(lambda x: np.prod(1 + x
/ 100) - 1)
df["Ret60_Low"] = df["Lo PRIOR"].rolling(60).apply(lambda x: np.prod(1 + x
/ 100) - 1)
df["Ret120_Low"] = df["Lo PRIOR"].rolling(120).apply(lambda x: np.prod(1 +
x / 100) - 1)
df["Ret240_Low"] = df["Lo PRIOR"].rolling(240).apply(lambda x: np.prod(1 +
x / 100) - 1)

df["Ret60"] = df["Ret60_MOMi"].shift(-60)
df = df.dropna()
df.tail(10)

df = df.drop(
    [
        "Lo PRIOR",
        "PRIOR 2",
        "PRIOR 3",
        "PRIOR 4",
        "PRIOR 5",
        "PRIOR 6",
        "PRIOR 7",
        "PRIOR 8",
        "PRIOR 9",
        "Hi PRIOR",
        "Mom",
    ],
    axis=1,
)

```

Finally, we get rid of the information in our dataframe that we are not going to use. Our final dataset looks like the following:

All rights reserved WQU
WorldQuant University QQQQ

In [5]: `df.head()`

Out[5]:

	Re t	Re t1 0	Re t2 5	Re t6 0	Re t1 20	Re t2 40	Re t1 0	Re t2 5	Re t6 0	Re t1 20	Re t2 40	Re t1 0	Re t2 5	Re t6 0	Re t1 20	Re t2 40	Re t6 0
		M	M	M	M	M	hi	hi	hi	h	h	Lo	Lo	Lo	L	L	
		O	O	O	O	O				i	i	w	w	w	w	w	
		Mi	Mi	Mi	Mi	Mi											
19	1.	-0.	-0.	0.	0.	0.	-0.	0.	0.	0.	0.	0.	0.	-0.	-0.	0.	0.
27	60	00	01	02	20	27	00	01	03	18	45	00	02	00	02	12	10
-0		71	16	95	15	58	38	09	14	17	83	33	19	01	10	51	26
8-		89	32	66	90	52	97	58	62	92	03	76	60	12	26	74	16
19																	
19	0.	-0.	-0.	0.	0.	0.	-0.	0.	0.	0.	0.	0.	0.	0.	-0.	0.	0.
27	74	01	00	02	19	26	00	02	04	19	45	00	02	01	00	13	08
-0		60	20	84	11	75	66	77	38	28	80	97	89	30	29	23	77
8-		77	22	43	85	48	44	04	32	65	15	04	20	91	74	84	30
20																	
19	0.	-0.	0.	0.	0.	0.	-0.	0.	0.	0.	0.	0.	0.	0.	-0.	0.	0.
27	71	00	00	03	21	27	00	02	05	20	45	00	02	01	01	12	08
-0		85	29	38	66	97	27	52	49	93	99	59	13	84	03	30	18
8-		96	57	84	75	47	75	56	55	46	03	75	49	98	11	46	98
22																	
19	0.	0.	0.	0.	0.	0.	0.	0.	0.	0.	0.	-0.	0.	0.	-0.	0.	0.
27	97	03	00	03	21	28	03	01	05	20	45	00	00	01	00	12	04
-0		31	81	89	34	03	15	86	67	78	93	15	94	52	88	20	02
8-		52	49	26	31	81	62	38	51	97	20	83	27	19	10	26	16
23																	
19	0.	0.	0.	0.	0.	0.	0.	0.	0.	0.	0.	0.	0.	0.	-0.	0.	0.
27	91	06	00	06	22	31	07	01	08	20	47	00	00	01	01	10	03
-0		19	79	67	55	45	06	54	65	80	00	76	64	63	86	10	67
8-		88	49	28	76	10	20	91	07	18	46	66	83	47	10	36	11
24																	

3.2 Train-Test Samples and Scaling

Our next step is selecting our training and test samples, as well as undertaking some pre-processing of information (e.g., scaling inputs). This should not be at all something new to you, but we are going to do it with the help of the *sklearn* library in Python.

To begin with, we make sure that our data is indexed appropriately, that we have a variable containing the dates, that we have data for all inputs (hence the final dataset will start in August 1927), and that all these changes are 'inplace':

```
In [6]: from sklearn.model_selection import train_test_split
```

```
df.reset_index(inplace=True)
df.rename(columns={"index": "Date"}, inplace=True)
df.head()
```

Out[6]:

	Date	Ret	Ret_1	Ret_2	Ret_6	Ret_1	Ret_2	Ret_1	Ret_2	Ret_6	Ret_1	Ret_2	Ret_1	Ret_2	Ret_6	Ret_1	Ret_2	Ret_6
			0_M	5_M	0_M	2_M	4_M	0_hi	5_hi	0_hi	2_hi	4_hi	0_Lo	5_Lo	0_Lo	2_Lo	4_Lo	0
			O	O	O	O	O				hi	hi	Lo	Lo	Lo	Lo	Lo	
			M	M	M	M	M											
			i	i	i	M	O									w	w	
						i	i											
0	1927-08-19	1.60	-0.007189	-0.011632	0.029566	0.021590	0.027582	-0.003897	0.000158	0.000362	0.01817	0.04583	0.003319	0.020112	-0.00026	0.021051	0.125174	0.102616
1	1927-08-20	0.74	-0.016077	-0.010022	0.028443	0.011185	0.026754	-0.000664	0.000774	0.00038	0.01928	0.04580	0.009789	0.028930	0.000191	-0.000297	0.132374	0.087784
2	1927-08-22	0.71	-0.008596	0.002957	0.033884	0.026675	0.027974	-0.000275	0.000525	0.00005	0.02049	0.04593	0.009959	0.021384	0.000101	-0.000303	0.123011	0.081898
3	1927-08-23	0.97	0.033152	0.008149	0.038926	0.023431	0.028031	0.001562	0.008638	0.00051	0.02078	0.04593	-0.001583	0.009427	0.000119	-0.0008810	0.122610	0.040216
4	1927-08-24	0.91	0.061988	0.007949	0.062876	0.022255	0.0314510	0.000620	0.005420	0.00065	0.02080	0.04700	0.007600	0.006463	0.000186	-0.0001086	0.101036	0.036711

Next, we will define the size of our **test sample**, using 40% of the observations in the whole sample as a test. We have to be careful in this regard when working in a financial market context. While this is not applicable in other settings, when it comes to predicting stock/portfolio returns, we need to make sure the chronological order in the training and test samples is followed. We cannot evaluate the performance of a model in test by selecting a sample of something that occurred before the training. It does not make sense; it would be like asking the model to predict something that already happened!

Thus, we will devote the first 60% of our sample (chronologically) to training and the most recent 40% to test (or, better said, the closest integer to that percentage). Note also that we have defined the return of the momentum portfolio as '**Ret**'. But remember this is **not** what we are trying to predict (that

would be 'Ret60'); we just want it there because it will be helpful later on to compute the returns from a buy-and-hold strategy for comparison purposes.

```
In [7]: df.reset_index(inplace=True, drop=True)

ts = int(0.4 * len(df)) # Number of observations in the test sample
split_time = len(df) - ts # From this data we are in the test sample
test_time = df.iloc[split_time:, 0:1].values # Keep the test sample dates
Ret_vector = df.iloc[split_time:, 1:2].values
df.tail()
```

Out[7]:

	D	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R
	at	et	et	et	et	et	et	et	et	et	et	et	et	et	et	et	et
	e		1	2	6	1	2	1	2	6	1	2	1	2	6	1	2
			0	5	0	2	4	0	5	0	2	4	0	5	0	2	4
			M	M	M	O	O	hi	hi	hi	0	0	Lo	Lo	Lo	0	0
			O	O	O	M	M				hi	hi	w	w	w	Lo	Lo
			M	M	M	O	O									w	w
			i	i	i	M	M										
						i	i										
2	20	1.	0.	0.	0.	0.	0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.
4	22	94	03	10	18	02	13	08	08	06	30	15	12	18	24	35	28
8	-0		87	85	86	84	10	69	75	14	32	23	60	85	41	37	53
8	4-		12	92	28	58	28	25	99	17	05	30	17	92	55	31	41
7	27																17
2	20	-0.	0.	0.	0.	0.	0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.
4	22	74	04	08	18	05	12	07	05	05	26	12	11	14	23	33	26
8	-0		36	99	24	18	29	40	76	20	66	87	78	76	27	50	03
8	4-		54	26	33	78	95	19	57	85	82	22	22	83	96	72	62
8	28																72
2	20	0.	0.	0.	0.	0.	0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.
4	22	41	02	08	10	05	12	09	11	09	29	15	12	19	21	35	28
8	-0		95	18	12	35	51	73	17	56	01	63	79	08	03	74	52
8	4-		05	45	72	57	24	48	44	52	64	69	62	62	33	85	94
9	29																55
2	20	-2.	-0.	0.	0.	0.	0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.
4	22	61	02	03	05	00	11	08	09	05	29	12	06	13	13	33	24
8	-0		59	01	03	94	05	80	50	02	00	42	93	47	04	00	92
9	5-		06	22	66	04	28	34	37	30	25	92	27	76	92	29	82
0	02																54
2	20	2.	0.	0.	0.	0.	0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.
4	22	04	02	07	11	04	12	08	08	04	27	13	10	15	17	33	26
8	-0		55	11	18	18	03	19	40	50	23	39	93	77	29	48	42
9	5-		52	67	19	73	00	55	83	39	33	64	07	15	54	60	72
1	03																70

Now, we can undertake our separation of **training** and **test** samples. Once again, note that we select **shuffle = False**, as these would negatively impact our chronological order issue, as previously mentioned:

```
In [8]: Xdf, ydf = df.iloc[:, 2:-1], df.iloc[:, -1]
X = Xdf.astype("float32")
y = ydf.astype("float32")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=ts, shuffle=False
) # It is important to keep "shuffle=False"
n_features = X_train.shape[1]
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)

(14936, 15) (9956, 15) (14936,) (9956,)
```

So, we are going to have a total of 14936 observations (examples) in our training set and 9956 obs in the test sample. Of course, these observations correspond to just 1 output ('Ret60') but to 15 different inputs (the 10, 25, 60, 120, and 240-day returns for the 'Mom', 'Hi' and 'Lo' portfolios).

The next step is **scaling**. As you probably remember, scaling the inputs and outputs using some transformation is important in order for the algorithm to handle different scales of the values. In this case, we will choose a **MinMaxScaler** in the range (-1, 1) for both output and input variables:

```
In [9]: # Scaling

from sklearn.preprocessing import MinMaxScaler

scaler_input = MinMaxScaler(feature_range=(-1, 1))
scaler_input.fit(X_train)
X_train = scaler_input.transform(X_train)
X_test = scaler_input.transform(X_test)

mean_ret = np.mean(y_train) # Useful to compute the performance = R2

scaler_output = MinMaxScaler(feature_range=(-1, 1))
y_train = y_train.values.reshape(len(y_train), 1)
y_test = y_test.values.reshape(len(y_test), 1)
scaler_output.fit(y_train)
y_train = scaler_output.transform(y_train)
y_test = scaler_output.transform(y_test)
```

3.3 Model and Training

Now we are ready to start training our model. We will do this using **Tensorflow** and the Keras framework. At this point, we will create a very simple model with a single layer, but the structure will be equivalent for more complex and deep networks as well:

- **Define the model and the number of layers:** We do this with the `tf.keras.layers.Dense(1)` command.
- **Define learning rate:** In this case, we consider a learning rate of $10^{-5} = 0.00001$

- **Define the optimizer to use:** In this case, we will select a very famous optimizer used in deep learning: Adam. We will get back to the Adam optimizer in the future. To learn more about how this works, you should go to section 12.10 in the book '*Dive into Deep Learning*' by Zhang et. al.
- **Define the loss function:** Here, as we are trying to mimic linear regression, we will select a mean squared error (MSE) loss function. In future notebooks, we will learn more about the appropriateness of mean squared error for financial applications.

```
In [11]: import tensorflow as tf

tf.random.set_seed(1)

model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Dense(1))

hp_lr = 1e-5

adam = tf.keras.optimizers.Adam(learning_rate=hp_lr)
model.compile(optimizer=adam, loss="mean_squared_error")
```

Once we have defined the model, let's put it to good use!

Next we fit the model on the training set data. When doing this, we will have to select:

- **Data (X, y) to be used:** Here, the training data for the inputs (X_train) and output (y_train).
- **Number of epochs to train the model:** Each epoch means "one lap around all the training set examples" when the parameters are all updated. In this case, we select 50 epochs.
- **Batch (or minibatch) size:** The size of the (mini)batch determines how many examples at a time we are going to consider in each iteration of our algorithm. Usually, to improve efficiency of the process, a batch size multiple of 32 is preferred.

Finally, the '**verbose=2**' option that you see in the code below is simply to show the progress of model training in-screen. You can actually consult all the documentation on the different options when training a model in Keras here: https://keras.io/api/models/model_training_apis/

```
In [12]: model.fit(X_train, y_train, epochs=50, batch_size=32, verbose=2)
```

Epoch 1/50
467/467 - 1s - 3ms/step - loss: 0.1515
Epoch 2/50
467/467 - 1s - 2ms/step - loss: 0.1410
Epoch 3/50
467/467 - 1s - 2ms/step - loss: 0.1316
Epoch 4/50
467/467 - 1s - 2ms/step - loss: 0.1230
Epoch 5/50
467/467 - 1s - 3ms/step - loss: 0.1152
Epoch 6/50
467/467 - 1s - 3ms/step - loss: 0.1082
Epoch 7/50
467/467 - 1s - 3ms/step - loss: 0.1019
Epoch 8/50
467/467 - 1s - 3ms/step - loss: 0.0962
Epoch 9/50
467/467 - 1s - 3ms/step - loss: 0.0912
Epoch 10/50
467/467 - 1s - 3ms/step - loss: 0.0868
Epoch 11/50
467/467 - 1s - 3ms/step - loss: 0.0830
Epoch 12/50
467/467 - 1s - 2ms/step - loss: 0.0796
Epoch 13/50
467/467 - 1s - 3ms/step - loss: 0.0768
Epoch 14/50
467/467 - 1s - 3ms/step - loss: 0.0744
Epoch 15/50
467/467 - 1s - 3ms/step - loss: 0.0725
Epoch 16/50
467/467 - 1s - 2ms/step - loss: 0.0708
Epoch 17/50
467/467 - 1s - 3ms/step - loss: 0.0695
Epoch 18/50
467/467 - 1s - 3ms/step - loss: 0.0685
Epoch 19/50
467/467 - 1s - 3ms/step - loss: 0.0676
Epoch 20/50
467/467 - 1s - 3ms/step - loss: 0.0669
Epoch 21/50
467/467 - 1s - 2ms/step - loss: 0.0662
Epoch 22/50
467/467 - 1s - 2ms/step - loss: 0.0657
Epoch 23/50
467/467 - 1s - 3ms/step - loss: 0.0651
Epoch 24/50
467/467 - 1s - 3ms/step - loss: 0.0646
Epoch 25/50
467/467 - 1s - 3ms/step - loss: 0.0641
Epoch 26/50
467/467 - 1s - 3ms/step - loss: 0.0636
Epoch 27/50
467/467 - 1s - 2ms/step - loss: 0.0631
Epoch 28/50
467/467 - 1s - 3ms/step - loss: 0.0626
Epoch 29/50
467/467 - 1s - 3ms/step - loss: 0.0621
Epoch 30/50
467/467 - 1s - 3ms/step - loss: 0.0617

```
Epoch 31/50
467/467 - 1s - 3ms/step - loss: 0.0612
Epoch 32/50
467/467 - 1s - 3ms/step - loss: 0.0607
Epoch 33/50
467/467 - 1s - 3ms/step - loss: 0.0603
Epoch 34/50
467/467 - 1s - 3ms/step - loss: 0.0598
Epoch 35/50
467/467 - 1s - 3ms/step - loss: 0.0594
Epoch 36/50
467/467 - 1s - 2ms/step - loss: 0.0590
Epoch 37/50
467/467 - 1s - 2ms/step - loss: 0.0585
Epoch 38/50
467/467 - 1s - 2ms/step - loss: 0.0581
Epoch 39/50
467/467 - 1s - 2ms/step - loss: 0.0577
Epoch 40/50
467/467 - 1s - 2ms/step - loss: 0.0573
Epoch 41/50
467/467 - 1s - 3ms/step - loss: 0.0569
Epoch 42/50
467/467 - 1s - 3ms/step - loss: 0.0565
Epoch 43/50
467/467 - 1s - 3ms/step - loss: 0.0561
Epoch 44/50
467/467 - 1s - 2ms/step - loss: 0.0557
Epoch 45/50
467/467 - 1s - 2ms/step - loss: 0.0553
Epoch 46/50
467/467 - 1s - 3ms/step - loss: 0.0549
Epoch 47/50
467/467 - 1s - 3ms/step - loss: 0.0546
Epoch 48/50
467/467 - 1s - 3ms/step - loss: 0.0542
Epoch 49/50
467/467 - 1s - 2ms/step - loss: 0.0538
Epoch 50/50
467/467 - 1s - 3ms/step - loss: 0.0535
```

```
Out[12]: <keras.src.callbacks.history.History at 0x78bd8e779b10>
```

So our model is trained!

Moving forward, one thing that will be important to understand with more complex models is the ***model.summary()*** feature. With it, we can get a quick sense of how the model is built, how many parameters it has, the shape of each of the layers, etc.

In this case, our model only had 1 dense layer and included 16 parameters to train. Let's take a look:

```
In [14]: model.summary()
```

Model: "sequential_1"

Layer (type) Param #	Output Shape	
dense_1 (Dense) 16	(None, 1)	

Total params: 50 (204.00 B)

Trainable params: 16 (64.00 B)

Non-trainable params: 0 (0.00 B)

Optimizer params: 34 (140.00 B)

4. Performance Measures in Finance

Now that we have trained our model parameters, let's check how our model predictive ability performs out of sample. We are going to check this in two ways:

- Measures of model fit (similar to the R-squared measure that we have already seen before).
- Performance of a trading strategy based on a model's predictions.

4.1. Measure of Model Fit in Finance

We already know that, in linear regression, a good measure of model in-sample fit is the R^2 measure. Just as a reminder, this is defined as:

$$R^2 = 1 - \frac{\sum (y - Xw)^2}{\sum (y - \bar{y})^2} = 1 - \frac{SSE}{TSE}$$

The in-sample R^2 will serve as a proxy of how good our model fits the training set examples. However, when constructing predictive models in finance, we are more interested in the performance out-of-sample. In this regard, Campbell and Thompson (2008) came out with a measure, R_{OS}^2 , that captures the degree to which a predictive model is able to explain stock returns out-of-sample:

$$R_{OS}^2 = 1 - \frac{\sum_{t=1}^T (r_t - \hat{r}_t)^2}{\sum_{t=1}^T (r_t - \bar{r}_t)^2}$$

where r_t is the actual return over period t , $\hat{r}_t$ is the predicted returns for t using trained model parameters in $t - 1$, and $\bar{r}_t$ is the mean historical return calculated over $t - 1$. You can see more on this discussion in the Campbell and Thompson paper:

- Campbell, John Y., and Samuel B. Thompson. "Predicting Excess Stock Returns out of Sample: Can Anything Beat the Historical Average?" *The Review of Financial Studies*, vol. 21, no. 4, 2008, pp. 1509-1531, <https://www.nber.org/papers/w11468>.

Consequently, we will apply this out-of-sample R^2 measure to our framework to see how our linear regression model is able to predict momentum returns over the next 60 days.

First, we will need to get the actual momentum $t + 60$ -day returns in the test sample, which requires un-transforming the scaled variable. Then, we will use the `model.predict()` feature to use the weights (w) that we obtained when training our model to calculate the predicted momentum returns in the test sample according to those weights. Similar to the previous case, we also need to de-scale our variables:

```
In [15]: values = scaler_output.inverse_transform(y_test)

y_pred = model.predict(X_test)
y_pred = scaler_output.inverse_transform(y_pred)
```

312/312 ————— 0s 1ms/step

What is the shape of our predictions? Well, it must be a vector of 1 column (predicted output) by 9956 rows (the number of examples in the test sample):

```
In [16]: y_pred.shape
```

```
Out[16]: (9956, 1)
```

Now we have all the ingredients to calculate the R^2_{OS} from Campbell and Thompson (remember that we calculated `mean_ret` before!):

```
In [17]: def R2_campbell(y_true, y_predicted, mean_ret):
    y_predicted = y_predicted.reshape((-1,))
    sse = sum((y_true - y_predicted) ** 2)
    tse = sum((y_true - mean_ret) ** 2)
    r2_score = 1 - (sse / tse)
    return r2_score

R2_Campbell = R2_campbell(values.flatten(), y_pred.flatten(), mean_ret)

print("R2 (Campbell): ", R2_Campbell)
```

```
R2 (Campbell): -0.48810577
```

Not very surprisingly, our model has a large negative R_{OS}^2 coefficient, indicating a very poor predictive power of the linear model out of sample.

4.2. Testing Model Performance via Backtesting

Although a poor R_{OS}^2 is already an indicator that the predictive power of the model is not going to be very good, it does not necessarily imply that our model is useless. For example, think about a predictive model that delivers a correct prediction 10% of the time and a wrong one 90% of the time. We may still want to follow this model in a trading strategy because it could happen that the 10% of the time the model is right, it delivers such a great return that it more than compensates the trading losses of being wrong. In short, by following the model, you lose small most of the time, and when you win, you win big. This could be an attractive trading strategy for some.

Therefore, next, we will take a look at the predicted versus real 60-day return of the Mom factor:

```
In [18]: df_predictions = pd.DataFrame(  
    {  
        "Date": test_time.flatten(),  
        "Pred": y_pred.flatten(),  
        "Ret": (Ret_vector.flatten() / 100),  
        "Values": values.flatten(),  
    }  
    )  
df_predictions.tail()
```

```
Out[18]:
```

	Date	Pred	Ret	Values
9951	2022-04-27	0.114044	0.0194	-0.041617
9952	2022-04-28	0.108484	-0.0074	-0.000872
9953	2022-04-29	0.097907	0.0041	-0.044455
9954	2022-05-02	0.090220	-0.0261	-0.008054
9955	2022-05-03	0.110699	0.0204	-0.010970

We also want to make sure that 'dates' are correctly expressed in the new DataFrame:

```
In [19]: df_predictions.Date = pd.to_datetime(df_predictions.Date, format="%YYYY-%mm-%dd")  
df = df_predictions  
df.tail()
```

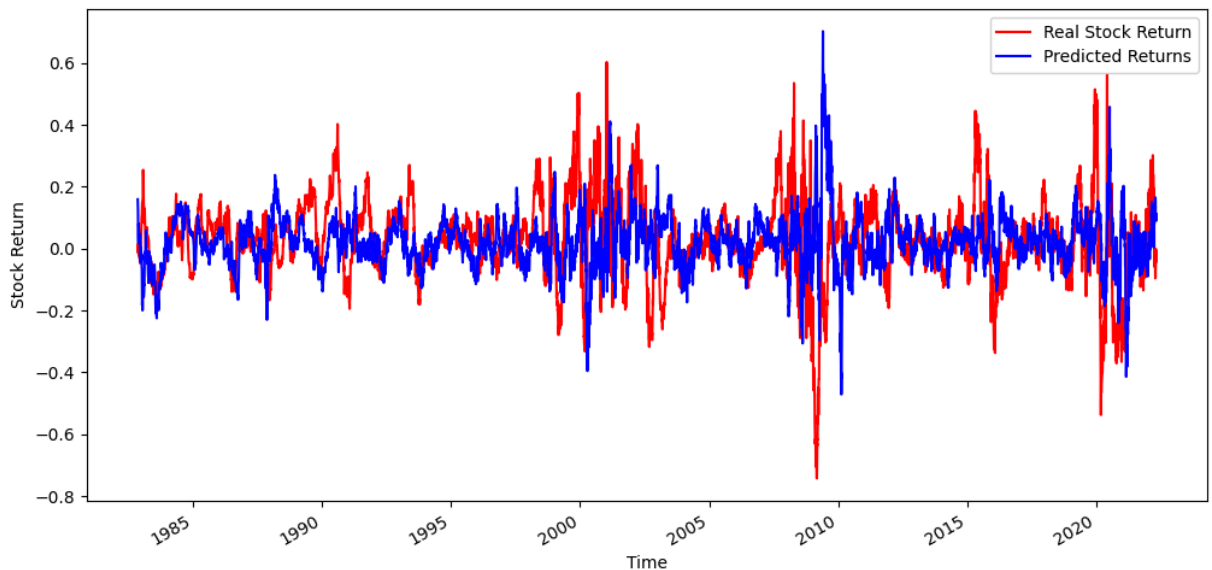
Out[19]:

	Date	Pred	Ret	Values
9951	2022-04-27	0.114044	0.0194	-0.041617
9952	2022-04-28	0.108484	-0.0074	-0.000872
9953	2022-04-29	0.097907	0.0041	-0.044455
9954	2022-05-02	0.090220	-0.0261	-0.008054
9955	2022-05-03	0.110699	0.0204	-0.010970

So let's see graphically what these two (predicted vs. real 60-day returns) look like.

In [20]: `import matplotlib.pyplot as plt`

```
fig = plt.figure(figsize=(12, 6))
ax = plt.gca()
df.plot(x="Date", y="Values", color="red", label="Real Stock Return",
ax=ax)
df.plot(x="Date", y="Pred", color="blue", label="Predicted Returns",
ax=ax)
plt.xlabel("Time")
plt.ylabel("Stock Return")
plt.legend()
plt.show()
```



From the previous graph we can observe that, at times, the predicted versus real Mom returns overlap, indicating that our model has some kind of prediction power. But of course, other times, the two go in completely opposite directions, which is not a very good sign.

Next, we will perform **Backtesting** on a very simple strategy using the predicted returns:

- **Strategy:**

Our **strategy** will simply consist of going long on the Mom portfolio on a given day when the model predicts a positive return, and going short when the model

predicts a negative one. Thus, if for a given day d our model predicts a positive return, we would have gone long the previous day (at close prices) so that we will earn the Mom factor returns for that day d (which we refer to as 'Ret' in the DataFrame). Conversely, if the model predicts a negative return, we will go short the previous day so that we will earn minus day d return ($-Ret_d$).

What we just described would be a classic long/short strategy. For comparison purposes, we also perform backtesting on a strategy that only goes long (omitting the short part in the previous strategy) and a Buy-and-hold strategy that simply invests in the Mom factor at the beginning of the test period and keeps the investment unaltered until the end of the period.

- **Backtesting:**

Backtesting is a very common way to evaluate a trading strategy. Intuitively, it consists of evaluating a trading strategy following the idea of '*how would I have done if I followed this strategy in a given period*'. For us, that period is the testing period. Following the strategy implies that at each point in time, we would be deciding with the information available just until that point in time. This is a very important condition we need to keep in mind because otherwise we would be contaminating our strategy and thus any inferences we draw on its validity.

Next, we calculate the positions that our strategy would keep each day in the backtest (that is 1 for long, -1 for short). By simply multiplying this *Positions* vector by the Mom factor daily returns (*Ret*), we can obtain the daily returns of our strategy. We can take a very similar approach to obtain daily returns of the long-only strategy.

Then, we calculate the cumulative returns of the different strategies (long/short, long-only, and buy-and-hold) using the *lambda* feature in Python, as well as the final values obtained for our investment by the end of the testing period:

```
In [21]: df["Positions"] = df["Pred"].apply(np.sign)
df["Strat_ret"] = df["Positions"].shift(1) * df["Ret"]
df["Positions_L"] = df["Positions"].shift(1)
df["Positions_L"][df["Positions_L"] == -1] = 0
df["Strat_ret_L"] = df["Positions_L"] * df["Ret"]
df["CumRet"] = df["Strat_ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)
df["CumRet_L"] = df["Strat_ret_L"].expanding().apply(lambda x: np.prod(1 + x) - 1)
df["bhRet"] = df["Ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)

Final_Return_L = np.prod(1 + df["Strat_ret_L"]) - 1
Final_Return = np.prod(1 + df["Strat_ret"]) - 1
Buy_Return = np.prod(1 + df["Ret"]) - 1

print("Strat Return Long Only =", Final_Return_L * 100, "%")
print("Strat Return =", Final_Return * 100, "%")
print("Buy and Hold Return =", Buy_Return * 100, "%")
```

```
/tmp/ipykernel_19/4012059038.py:4: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in
certain cases, but when using Copy-on-Write (which will become the default
behaviour in pandas 3.0) this will never work to update the original
DataFrame or Series, because the intermediate object on which we are setting
values will behave as a copy.
A typical example is when you are setting values in a column of a DataFrame,
like:
```

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df["Positions_L"][df["Positions_L"] == -1] = 0
/tmp/ipykernel_19/4012059038.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df["Positions_L"][df["Positions_L"] == -1] = 0
```

```
Strat Return Long Only = 1934.1467909416804 %
```

```
Strat Return = 491.6320380719581 %
```

```
Buy and Hold Return = 1909.8411102470113 %
```

Judging by the numbers, it is clear that we are not doing a very good job in timing the momentum factor. As before, let's see this graphically:

```
In [ ]: import matplotlib.pyplot as plt

fig = plt.figure(figsize=(12, 6))
ax = plt.gca()
df.plot(x="Date", y="bhRet", label="Buy&Hold", ax=ax)
df.plot(x="Date", y="CumRet_L", label="Strat Only Long", ax=ax)
df.plot(x="Date", y="CumRet", label="Strat Long/Short", ax=ax)
plt.xlabel("date")
plt.ylabel("Cumulative Returns")
plt.grid()
plt.show()

df.describe()
```

So it would have clearly been better for us to simply have invested in the Mom factor portfolio and let our money stay there until the end of the period!

The previous table contains a bunch of different statistics for the different strategies that corroborate this idea. Notice that here we are simply looking at measures based on the distribution of returns according to each strategy followed. In practice, there are many more measures to assess the performance of a strategy in backtesting.

You are probably familiar with most of these measures, like *Sharpe Ratio*, *Kurtosis*, etc. There are many more such as *Maximum DrawDown (MaxDD)*,

Calmar Ratio, *Sortino Ratio*, etc. that we will use in future implementations of our neural network algorithms.

If you want to learn more about these at this point, here, you have a number of (extra) resources (there are many more easily found online):

- **Backtesting basics:** <https://medium.com/auquan/backtesting-basics-understanding-your-key-metrics-ed902c24c702>
- **Some trading strategy performance metrics** (allegedly top metrics, but with a very narrow focus): <https://improve-your.trade/blog/top-17-trading-metrics>
- **Trading system and strategy performance metrics:** <https://www.quantifiedstrategies.com/trading-strategy-and-system-performance-metrics/>
- **12 ways to measure trading performance:** <https://www.axi.com/int/blog/education/measure-your-trading-performance>
- **5 key backtesting metrics and implementation:** <https://blankly.finance/list-of-performance-metrics/>

5. Conclusion

In this lesson, we have used a very simple linear regression model to (i) introduce the framework of `Tensorflow` Keras, which will be where we develop all our future complex Neural Networks, and (ii) learn how to create a simple trading strategy and perform backtesting to evaluate its performance.

Sadly, our trading strategy based on linear regression delivered very poor results both in terms of performance and predictive power of the NN model. In the remaining lessons of the module, we will build a more complex NN and check how its predictive ability helps improve our trading strategy.

The first step for this is introducing multilayer perceptron (MLP) networks, which we will do in lesson 3. See you there!

References

- AQR. "Systematic Equities: A Closer Look." <https://www.aqr.com/Learning-Center/Systematic-Equities/Systematic-Equities-A-Closer-Look>.
- Asness, Clifford S., et al. "Value and Momentum Everywhere." *The Journal of Finance*, vol. 68, no. 3, 2013, pp. 929-985, [https://onlinelibrary.wiley.com/doi/abs/10.1111/jofi.12021?](https://onlinelibrary.wiley.com/doi/abs/10.1111/jofi.12021?casa_token=vxQDrj_UikoAAAAA:hXo8tu-)

G8xQb6CxRiS1C2hk8zLyYM6rlYrl3E8DqUnTY2TG1m_F2me8XUChr9S88W-csmUczqZQ44E8.

- Auquan. "Backtesting Basics: Understanding Your Key Metrics." *Medium*, 21 July 2020, <https://medium.com/auquan/backtesting-basics-understanding-your-key-metrics-ed902c24c702>.
- BlackRock. "Factor-Based Investing." BlackRock, Inc. <https://www.blackrock.com/uk/professionals/solutions/factor-based-investing>.
- Campbell, John Y., and Samuel B. Thompson. "Predicting Excess Stock Returns out of Sample: Can Anything Beat the Historical Average?" *The Review of Financial Studies*, vol. 21, no. 4, 2008, pp. 1509-1531.
- Cutkovic, Milan. "12 Ways to Measure Your Trading Performance." Axi, 28 September 2021, <https://www.axi.com/int/blog/education/measure-your-trading-performance>.
- Daniel, Kent, and Tobias J. Moskowitz. "Momentum Crashes." *Journal of Financial Economics*, vol. 122, no. 2, 2016, pp. 221-247, <https://www.sciencedirect.com/science/article/pii/S0304405X16301490>.
- Ehsani, Sina, and Juhani T. Linnainmaa. "Factor Momentum and the Momentum Factor." *The Journal of Finance*, vol. 77, no. 3, 2022, pp. 1877-1919, https://onlinelibrary.wiley.com/doi/full/10.1111/jofi.13131?casa_token=oWY7-8sCYqkAAAAA%3A-HrzsyemLuXf0lQyYMT9vxFNyNQugJsMfaY39EAeTCjCIDIAPHdU_fqS4z2-LpVrUO7li2wQmIphYXA.
- Erwin. "The Top 17 Trading Metrics (and Why You Should Care)." *EB Worx*, <https://improve-your.trade/blog/top-17-trading-metrics>.
- Fan, Brandon. "Trading Algos - 5 Key Backtesting Metrics and How to Implement Them." Blankly, 7 Jan. 2022, <https://blankly.finance/list-of-performance-metrics/>.
- Fama, Eugene F., and Kenneth R. French. "Common Risk Factors in the Returns on Stocks and Bonds." *Journal of Financial Economics*, vol. 33, no. 1, 1993, pp. 3-56, ([https://doi.org/10.1016/0304-405X\(93\)90023-5](https://doi.org/10.1016/0304-405X(93)90023-5)).
- Fama, Eugene F., and Kenneth R. French. "The Cross-Section of Expected Stock Returns." *The Journal of Finance*, vol. 47, no. 2, 1992, pp. 427-465, <https://www.jstor.org/stable/2329112>.
- French, Kenneth R. "Current Research Returns." https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html.
- French, Kenneth R. "Detail for 10 Returns Formed Daily on Momentum." https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/Data_Library/det_10_port_form_pr_12_2_daily.html.

- Harvey, Campbell R., et al. "... and the Cross-Section of Expected Returns." *The Review of Financial Studies*, vol. 29, no. 1, 2016, pp. 5-68.
- Huang, Dashan, et al. "Time Series Momentum: Is It There?" *Journal of Financial Economics*, vol. 135, no. 3, 2020, pp. 774-794, <https://core.ac.uk/download/pdf/287750773.pdf>.
- Jegadeesh, Narasimhan, and Sheridan Titman. "Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency." *The Journal of Finance*, vol. 48, no. 1, 1993, pp. 65-91.
- Moskowitz, Tobias J., et al. "Time Series Momentum." *Journal of Financial Economics*, vol. 104, no. 2, 2012, pp. 228-250, <https://www.sciencedirect.com/science/article/pii/S0304405X11002613>.
- Mullins, Jr., David W. "Does the Capital Asset Pricing Model Work?" *Harvard Business Review*, 1982, <https://hbr.org/1982/01/does-the-capital-asset-pricing-model-work>.
- Nielson, Darby, et al. *An Overview of Factor Investing: The Merits of Factors as Potential Building Blocks for Portfolio Construction*. Fidelity Investments, 2016.
- Quantified Strategies for Traders. "Trading System and Strategy Performance Metrics (What Is It and How to Use It)." 12 October 2022, <https://www.quantifiedstrategies.com/trading-strategy-and-system-performance-metrics/>.
- Vanguard. "Factor-Based Strategies." The Vanguard Group, Inc. <https://advisors.vanguard.com/investments/approach/factor-based-strategies>.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Zhang, Ashton, et al. *Dive into Deep Learning*. <https://d2l.ai/d2l-en.pdf>.

- **Required pages:** Chapter 5, section 5.1
- **Estimated reading time:** 15 minutes

2. Zhang, Ashton, et al. *Dive into Deep Learning*. <https://d2l.ai/d2l-en.pdf>.

- **Required pages:** Chapter 5, section 5.2
- **Estimated reading time:** 15 minutes

3. Zhang, Ashton, et al. *Dive into Deep Learning*. <https://d2l.ai/d2l-en.pdf>.

- **Required pages:** Chapter 5, section 5.3
- **Estimated reading time:** 30 minutes

Module 7: Lesson 3

Multilayer Perceptrons (MLP)



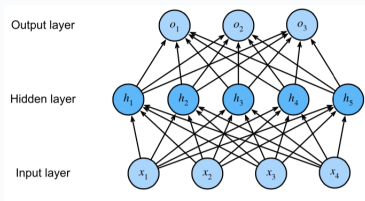
Outline

- ▶ Multilayer Perceptrons (MLP) and hidden layers
- ▶ Activation functions: ReLU, sigmoid, tanh
- ▶ Forward and Backward propagation
- ▶ Early-stopping

Multilayer perceptron (MLP)

Now that we know how to implement linear regression as a neural network in Tensorflow, let's introduce a truly deep model. We will do so using the simplest type of deep networks: **multilayer perceptron (MLP)**.

One of the main features of any deep network is a number of hidden layers. These layers, as we will see shortly, will allow us to explore non-linear relationships in the data:

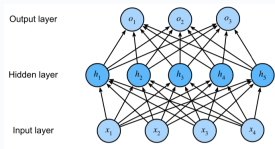


- How does data go from the input to output layer (O_q)?

$$H = XW^{(1)} + b^{(1)}$$

$$O = HW^{(2)} + b^{(2)}$$

Multilayer perceptron (MLP)



- What are the dimensions of each of these terms?
 - X is a matrix of $n \times d$ (a minibatch of n examples for d inputs)
 - $W^{(1)}$ is a matrix of $d \times h$ (d inputs for h units in the hidden layer). Here $h = 5$
 - $b^{(1)}$ is a vector of $1 \times h$ (1 bias term for each unit in hidden layer)
 - H is a matrix of $(n \times d)(d \times h) + (1 \times h) = n \times h$ (a minibatch of n examples for h hidden-layer units)
 - $W^{(2)}$ is a matrix of $h \times q$ (h units in the hidden layer by q units in the output layer). Here $q = 3$
 - $b^{(2)}$ is a vector of $1 \times q$ (1 bias term for each unit in output layer)
 - O is a matrix of $(n \times h)(h \times q) + (1 \times q) = n \times q$

Activation functions

Now, if all these layers are fully connected and always active, there is little actual gain from the linear regression case → **where is the non-linearity?**

Our secret ingredient is going to be a **non-linear activation function**. These functions transform inputs, in a non-linear way, to decide whether neurons of a layer are activated (or not).

- Thus, for an activation function $\sigma()$, our previous set of equations will look like:

$$H = \sigma \left(XW^{(1)} + b^{(1)} \right)$$

$$O = HW^{(2)} + b^{(2)}$$

The previous example had 1 hidden layer. If more, we could apply a (different) activation function to each one:

$$H^{(1)} = \sigma_1 \left(XW^{(1)} + b^{(1)} \right)$$

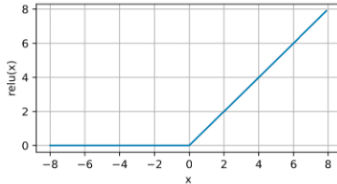
$$H^{(2)} = \sigma_2 \left(H^{(1)} W^{(2)} + b^{(2)} \right)$$

$$O = H^{(2)} W^{(3)} + b^{(3)}$$

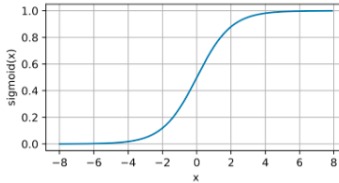
Activation functions

Since it is actually the activation function that ultimately brings non-linearity to our deep networks, it seems reasonable to review the main one used in practice:

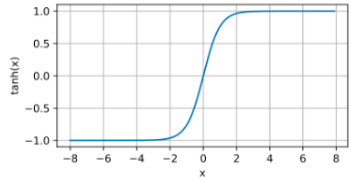
$$\text{ReLU}(x) = \max(x, 0).$$



$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}.$$



$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}.$$



Forward and Backward propagation

There are 2 key algorithms involved in the training of our neural networks: **forward and backward propagation**.

► **Forward propagation (or forward pass):**

- Refers to the calculation (and storage) of the intermediate variables (and outputs) of the network, in layer-to-layer order, from input to output layers.

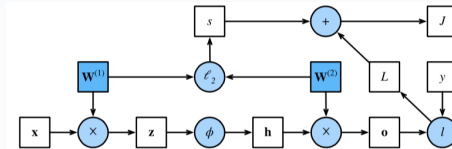


Fig. 5.3.1: Computational graph of forward propagation.

where, for a one hidden-layer network and bias term $b = 0$:

$z = W^{(1)}x \rightarrow h = \phi(z) \rightarrow o = W^{(2)}h \rightarrow$ and J is the regularized-cost function using an ℓ_2 norm.

Forward and Backward propagation

Forward prop is highly interconnected with **backward propagation**:

► **Backward propagation (backpropagation):**

- Refers to the method of calculating gradient of NN parameters, and updating them.
- The method essentially consists of calculating the partial derivative of the cost function with respect to the different parameters (i.e., $W^{(l)}$).
- To achieve this, it relies on the chain rule:

$$\frac{\partial Z}{\partial X} = \text{prod} \left(\frac{\partial Z}{\partial Y}, \frac{\partial Y}{\partial X} \right)$$

- The backprop algorithm will start with the gradient of the loss function and end obtaining the gradient of the model weights closest to the input layer (i.e., $W^{(1)}$):

$$\frac{\partial J}{\partial W^{(1)}} = \text{prod} \left(\frac{\partial J}{\partial z}, \frac{\partial z}{\partial W^{(1)}} \right) + \text{prod} \left(\frac{\partial J}{\partial s}, \frac{\partial s}{\partial W^{(1)}} \right) = \frac{\partial J}{\partial z} x^T + \lambda W^{(1)}$$

Early stopping

The forward prop and backprop algorithms will effectively train the model using iterative process:

- ▶ When do iterations/training stop?

The easy answer to this question is **when the stopping criteria is met**. But this begs another question:

- ▶ What is the stopping criteria?

There are many possible choices. For now, we are going to see one of the options: **Early Stopping**.

- ▶ Early stopping serves as both a stopping criterion and a regularization technique.
- ▶ Essentially, it consists of constraining the number of epochs of training. **How?**
 - During training, monitor the error in validation set (usually checking after each epoch).
 - **Stopping criteria:** stop when the validation error has not decreased by more than some amount ϵ for some number of epochs
 - How many is some epochs? → **patience**
 - Importantly, the **validation set** must NOT be a part of training nor test sets!

Summary of Lesson 3

In Lesson 3, we have looked at:

- ▶ Multilayer Perceptron (MLP) and hidden layers
- ▶ Activation functions: ReLU, sigmoid, tanh
- ▶ Forward and Backward propagation
- ▶ Early-stopping

⇒ **References for this lesson:**

Chapter 5, Sections 5.1, 5.2, and 5.3 of [Zhang, Aston, et al. *Dive into Deep Learning*. 2021.](#)

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement our first deep network (MLP) using Tensorflow for our trading strategy aimed at timing the momentum factor.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Zhang, Ashton, et al. *Dive into Deep Learning*. <https://d2l.ai/d2l-en.pdf>.

- **Required pages:** Chapter 5, section 5.5.3 on EarlyStopping
- **Estimated reading time:** 10 minutes

Module 7: Lesson 4

Momentum Timing with MLPs



Outline

- ▶ Constructing an MLP model
- ▶ MLPs for factor timing
- ▶ Backtesting the 'Mom timing' strategy

MLP model

This lesson will revisit a lot of the fundamental finance concepts of Lesson 2, where we deployed linear regression to develop a trading strategy based on momentum factor timing.

Now, however, we are going to build our first deep learning model using MLPs. In this case, we will choose a model with 4 layers:

- ▶ 3 Hidden layers with 50 neurons/units, 30 units, and 10 units, respectively
- ▶ Single-unit output layer

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 50)	800
dense_1 (Dense)	(None, 30)	1530
dense_2 (Dense)	(None, 10)	310
dense_3 (Dense)	(None, 1)	11

```
=====  
Total params: 2,651  
Trainable params: 2,651  
Non-trainable params: 0  
=====
```

Training the MLP model

Training the MLP model is going to be a little bit more complicated. We will have to make choices on:

- ▶ "Usual suspects": optimizer (Adam), learning rate (10^{-5}), loss function (MAE)
- ▶ Activation functions ('ReLU' for all units)
- ▶ EarlyStopping: stopping criteria, patience,...

```
act_fun = 'relu' # Activation function
hp_units = 50    # Units in the first hidden layer
hp_units_2 = 30  # Units in the second hidden layer
hp_units_3 = 10  # Units in the third hidden layer

model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Dense(units = hp_units, activation=act_fun))
model.add(tf.keras.layers.Dense(units = hp_units_2, activation=act_fun))
model.add(tf.keras.layers.Dense(units = hp_units_3, activation=act_fun))
model.add(tf.keras.layers.Dense(1))

hp_lr = 1e-5

adam = tf.keras.optimizers.Adam(lr=hp_lr)
model.compile(optimizer = adam, loss= 'mean_absolute_error')
es = tf.keras.callbacks.EarlyStopping(monitor='val_loss', mode='min', verbose=1, patience=10, restore_best_weights=True)
model.fit(X_train, y_train, validation_split=0.2, epochs=100, batch_size=32, verbose=2, callbacks=[es])
```

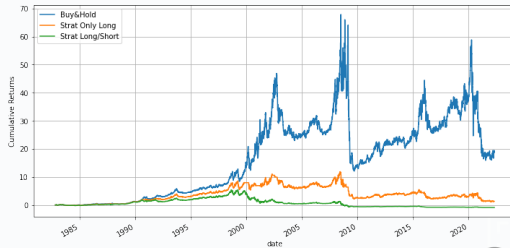
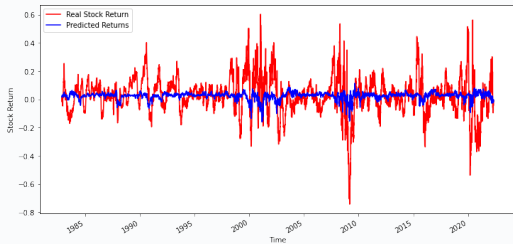
Performance of MLP model

As in previous cases, we will look at the performance of the model for predictive purposes by analyzing:

- Out-of-sample predictive power: E.g., $R^2_{OS} = 2.19\%$

Then, we can assess the viability of the trading strategy:

- Evaluating predicted returns vs. real ones in test sample (Backtest):



Summary of Lesson 4

In Lesson 4, we learned how to:

- ▶ Build an MLP model in Tensorflow-Keras
- ▶ Train the MLP model using features like EarlyStopping
- ▶ Backtest a trading strategy based on MLPs

⇒ **References for this Lesson:**

Chapter 5, Section 5.5.3. of [Zhang, Aston, et al. *Dive into Deep Learning*. 2021.](#)

TO DO NEXT: Go to this lesson's associated Jupyter Notebook. There, you will work on your first neural network model using MLPs to develop a momentum timing trading strategy.

This has been all for the Machine Learning course. We will dig much deeper into the deep learning models introduced in this last module in the next Deep Learning for Finance course. See you there!

MACHINE LEARNING

MODULE 8 | LESSON 4

Multilayer Perceptron: Timing Factors and Smart-Beta Strategies

Reading Time	50 minutes
Prior Knowledge	Linear Regression, Neural Network, Machine Learning
Keywords	Multilayer Perceptron, Deep learning, Neural Network, Momentum

In this lesson, we will revisit the momentum timing strategy of Lesson 2 using our newly acquired knowledge of neural networks. Multilayer perceptron networks (MLPs) will add non-linearity considerations to the prediction model, which will likely enhance its power-but let's check by how much.

1. MLP for Financial Market Predictions

Multilayer perceptrons (MLPs) are one of the simplest forms of neural networks and thus have been widely used for many applications, including the prediction of financial market returns. There have been several applications, including in the academic literature, exploiting the potential of MLPs. Here, we share a few of these works (you can easily find more online) for you to check in case you are interested. In the forthcoming Deep Learning in Finance course, we will review many more models and applications, but the works presented next could be used as a reasonable baseline from which to grow our models. As you will see, most of them do not really achieve strong return predictability:

- Guresen, Erkam, et al. "Using Artificial Neural Network Models in Stock Market Index Prediction." *Expert Systems with Applications*, vol. 38, no. 8, 2011), pp. 10389-10397, <https://www.sciencedirect.com/science/article/abs/pii/S0957417411002740?via%3Dihub>
- Rather, Akhter Mohiuddin, et al. "Recurrent Neural Network and a Hybrid Model for Prediction of Stock Returns." *Expert Systems with Applications* vol. 42, no. 6, 2015, pp. 3234-3241, <https://www.sciencedirect.com/science/article/abs/pii/S0957417414007684>

- Devadoss, A. Victor, and T. Antony Alphonse Liori. "Forecasting of Stock Prices using Multi Layer Perceptron." *International Journal of Computing Algorithm*, vol. 2, no. 1, 2013, pp. 440-449, http://ijwebt.com/abstract_meta_author.php?id=V2-I2-P6
- Namdari, Alireza, and Zhaojun Steven Li. "Integrating Fundamental and Technical Analysis of Stock Market through Multi-Layer Perceptron." 2018 IEEE Technology and Engineering Management Conference (TEMSCON). *IEEE*, 2018, https://ieeexplore.ieee.org/abstract/document/8488440?casa_token=hbO9qp7fjdMAAAAA:QsTb7IcWieVNRxCzuw1O57Pq23SbKcmEQ652cULn2enCiDbt9fGx2XKkPVDc9mAK7V8abX5y
- Anand, C. "Comparison of Stock Price Prediction Models using Pre-Trained Neural Networks." *Journal of Ubiquitous Computing and Communication Technologies*, vol. 3, no. 2, 2021, pp. 122-134, <https://irojournals.com/jucct/article/view/3/2/5>

Let's see how our MLP model is able to help in the prediction of momentum factor returns that we tackled in Lesson 2 using linear regression.

2. Timing Momentum with Multilayer Perceptron (MLPs)

2.1. Data

First, we proceed to load and treat all the necessary data to construct our first deep learning model. The different steps undertaken are virtually the same as the ones from the linear regression case from Lesson 2. We refer you to that notebook in case you have doubts on any of the steps:

```
In [1]: import numpy as np
import pandas as pd
```

```
In [2]: route = "10_Portfolios_Prior_12_2_Daily.csv"
```

```
In [3]: # Read the csv file again with skipped rows
df = pd.read_csv("10_Portfolios_Prior_12_2_Daily.csv", index_col=0)
# Format the date index
df.index = pd.to_datetime(df.index, format="%Y%m%d")
# Build de MOM strategy: Long "Hi PRIOR" and Short "Lo PRIOR"
df["Mom"] = df["Hi PRIOR"] - df["Lo PRIOR"]
df.head()
```

Out[3]:

	Lo PRI OR	PRI OR 2	PRI OR 3	PRI OR 4	PRI OR 5	PRI OR 6	PRI OR 7	PRI OR 8	PRI OR 9	Hi PRI OR	Mo m
192 6- 11- 03	-0.12	0.60	-0.09	0.30	-0.51	-0.22	-0.12	0.50	0.13	1.28	1.40
192 6- 11- 04	0.65	1.82	1.34	0.61	1.01	0.64	0.82	0.44	0.48	0.40	-0.25
192 6- 11- 05	-0.84	-0.77	-0.22	-0.15	-0.02	-0.02	-0.07	0.36	0.20	0.08	0.92
192 6- 11- 06	1.03	0.28	0.24	0.40	0.19	0.64	0.10	0.10	0.39	-0.68	-1.71
192 6- 11- 08	-0.06	0.11	1.78	0.28	0.36	0.23	0.30	1.17	0.58	-0.18	-0.12

- **Inputs and outputs**

```

In [4]: df["Ret"] = df["Mom"]
df["Ret10_MOMi"] = df["Mom"].rolling(10).apply(lambda x: np.prod(1 + x /
100) - 1)
df["Ret25_MOMi"] = df["Mom"].rolling(25).apply(lambda x: np.prod(1 + x /
100) - 1)
df["Ret60_MOMi"] = df["Mom"].rolling(60).apply(lambda x: np.prod(1 + x /
100) - 1)
df["Ret120_MOMi"] = df["Mom"].rolling(120).apply(lambda x: np.prod(1 + x /
100) - 1)
df["Ret240_MOMi"] = df["Mom"].rolling(240).apply(lambda x: np.prod(1 + x /
100) - 1)

df["Ret10_hi"] = df["Hi PRIOR"].rolling(10).apply(lambda x: np.prod(1 + x
/ 100) - 1)
df["Ret25_hi"] = df["Hi PRIOR"].rolling(25).apply(lambda x: np.prod(1 + x
/ 100) - 1)
df["Ret60_hi"] = df["Hi PRIOR"].rolling(60).apply(lambda x: np.prod(1 + x
/ 100) - 1)
df["Ret120_hi"] = df["Hi PRIOR"].rolling(120).apply(lambda x: np.prod(1 +
x / 100) - 1)
df["Ret240_hi"] = df["Hi PRIOR"].rolling(240).apply(lambda x: np.prod(1 +
x / 100) - 1)

df["Ret10_Low"] = df["Lo PRIOR"].rolling(10).apply(lambda x: np.prod(1 + x
/ 100) - 1)
df["Ret25_Low"] = df["Lo PRIOR"].rolling(25).apply(lambda x: np.prod(1 + x
/ 100) - 1)
df["Ret60_Low"] = df["Lo PRIOR"].rolling(60).apply(lambda x: np.prod(1 + x
/ 100) - 1)
df["Ret120_Low"] = df["Lo PRIOR"].rolling(120).apply(lambda x: np.prod(1 +
x / 100) - 1)
df["Ret240_Low"] = df["Lo PRIOR"].rolling(240).apply(lambda x: np.prod(1 +
x / 100) - 1)

df["Ret60"] = df["Ret60_MOMi"].shift(-60)
df = df.dropna()
df.tail(10)

df = df.drop(
    [
        "Lo PRIOR",
        "PRIOR 2",
        "PRIOR 3",
        "PRIOR 4",
        "PRIOR 5",
        "PRIOR 6",
        "PRIOR 7",
        "PRIOR 8",
        "PRIOR 9",
        "Hi PRIOR",
        "Mom",
    ],
    axis=1,
)

```

```

In [5]: df.head()

```

Out[5]:

	Re t	Re t1 0 M O Mi	Re t2 5 M O Mi	Re t6 0 M O Mi	Re t1 20 M O Mi	Re t2 40 M O Mi	Re t1 0 hī	Re t2 5 hī	Re t6 0 hī	Re t1 20 -h i	Re t2 40 -h i	Re t1 0 Lo w	Re t2 5 Lo w	Re t6 0 Lo w	Re t1 20 -L o w	Re t2 40 -L o w	Re t6 0
19 27 -0 8- 19	1. 60	-0. 00 71 89	-0. 01 16 32	0. 02 95 66	0. 20 15 90	0. 27 58 52	-0. 00 38 97	0. 01 09 58	0. 03 14 62	0. 18 17 92	0. 45 83 03	0. 00 33 76	0. 02 19 60	-0. 00 01 12	-0. 02 10 26	0. 12 51 74	0. 10 26 16
19 27 -0 8- 20	0. 74	-0. 01 60 77	-0. 00 20 22	0. 02 84 43	0. 19 11 85	0. 26 75 48	-0. 00 66 44	0. 02 77 04	0. 04 38 32	0. 19 28 65	0. 45 80 15	0. 00 97 04	0. 02 89 20	0. 01 30 91	-0. 00 29 74	0. 13 23 84	0. 08 77 30
19 27 -0 8- 22	0. 71	-0. 00 85 96	0. 00 29 57	0. 03 38 84	0. 21 66 75	0. 27 97 47	-0. 00 27 75	0. 02 52 56	0. 05 49 55	0. 20 93 46	0. 45 99 03	0. 00 59 75	0. 02 13 49	0. 01 84 98	-0. 01 03 11	0. 12 30 46	0. 08 18 98
19 27 -0 8- 23	0. 97	0. 03 31 52	0. 00 81 49	0. 03 89 26	0. 21 34 31	0. 28 03 81	0. 03 15 62	0. 01 86 38	0. 05 67 51	0. 20 78 97	0. 45 93 20	-0. 00 15 83	0. 00 94 27	0. 01 52 19	-0. 00 88 10	0. 12 20 26	0. 04 02 16
19 27 -0 8- 24	0. 91	0. 06 19 88	0. 00 79 49	0. 06 67 28	0. 22 55 76	0. 31 45 10	0. 07 06 20	0. 01 54 91	0. 08 65 07	0. 20 80 18	0. 47 00 46	0. 00 76 66	0. 00 64 83	0. 01 63 47	-0. 01 86 10	0. 10 10 36	0. 03 67 11

• Train-Test samples and Scaling

```
In [6]: from sklearn.model_selection import train_test_split

df.reset_index(inplace=True)
df.rename(columns={"index": "Date"}, inplace=True)
df.head()
```

Out[6]:

	D ate	R et	R et 1 0 M O M i	R et 2 5 M O M i	R et 6 0 M O M i	R et 1 2 4 0 M O M i	R et 2 1 4 0 M O M i	R et 1 0 h i	R et 2 5 h i	R et 6 0 h i	R et 1 2 4 0 h i	R et 2 1 4 0 h i	R et 1 0 L o w	R et 2 5 L o w	R et 6 0 L o w	R et 1 2 4 0 L o w	R et 2 1 4 0 L o w	R et 6 0
0	19 27 -0 8- 19	1. 60	-0. 00 71 89	-0. 01 16 32	0. 02 95 66	0. 20 15 90	0. 27 58 52	-0. 00 38 97	0. 01 09 58	0. 03 14 62	0. 18 17 92	0. 45 83 03	0. 00 33 76	0. 02 19 60	-0. 00 01 12	-0. 02 10 26	0. 12 51 74	0. 10 26 16
1	19 27 -0 8- 20	0. 74	-0. 01 60 77	-0. 00 20 22	0. 02 84 43	0. 19 11 85	0. 26 75 48	-0. 00 66 44	0. 02 77 04	0. 04 38 32	0. 19 28 65	0. 45 80 15	0. 00 97 04	0. 02 89 20	0. 01 30 91	-0. 00 29 74	0. 13 23 84	0. 08 77 30
2	19 27 -0 8- 22	0. 71	-0. 00 85 96	0. 00 29 57	0. 03 38 84	0. 21 66 75	0. 27 97 47	-0. 00 27 75	0. 02 52 56	0. 05 49 55	0. 20 93 46	0. 45 99 03	0. 00 59 75	0. 02 13 49	0. 01 84 98	-0. 01 03 11	0. 12 30 46	0. 08 18 98
3	19 27 -0 8- 23	0. 97	0. 03 31 52	0. 00 81 49	0. 03 89 26	0. 21 34 31	0. 28 03 81	0. 03 15 62	0. 01 86 38	0. 05 67 51	0. 20 78 97	0. 45 93 20	-0. 00 15 83	0. 00 94 27	0. 01 52 19	-0. 00 88 10	0. 12 20 26	0. 04 02 16
4	19 27 -0 8- 24	0. 91	0. 06 19 88	0. 00 79 49	0. 06 67 28	0. 22 55 76	0. 31 45 10	0. 07 06 20	0. 01 54 91	0. 08 65 07	0. 20 80 18	0. 47 00 46	0. 00 76 66	0. 00 64 83	0. 01 63 47	-0. 01 86 10	0. 10 10 36	0. 03 67 11

In [7]: `df.reset_index(inplace=True, drop=True)`

```
ts = int(0.4 * len(df)) # Number of observations in the test sample
split_time = len(df) - ts # From this data we are in the test sample
test_time = df.iloc[split_time:, 0:1].values # Keep the test sample dates
Ret_vector = df.iloc[split_time:, 1:2].values
df.tail()
```

Out[7]:

	D at e	R et	R et 1 0 M O M i	R et 2 5 M O M i	R et 6 0 M O M i	R et 1 2 4 0 M O M i	R et 2 4 0 M O M i	R et 1 0 h i	R et 2 5 h i	R et 6 0 h i	R et 1 2 4 0 h i	R et 2 4 0 h i	R et 1 0 L o w	R et 2 5 L o w	R et 6 0 L o w	R et 1 2 4 0 L o w	R et 2 4 0 L o w	R et 6 0
2	20	1.	0.	0.	0.	0.	0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.
4	22	94	03	10	18	02	13	08	08	06	30	15	12	18	24	35	28	04
8	-0		87	85	86	84	10	69	75	14	32	23	60	85	41	37	53	16
8	4-		12	92	28	58	28	25	99	17	05	30	17	92	55	31	41	17
7	27																	
2	20	-0.	0.	0.	0.	0.	0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.
4	22	74	04	08	18	05	12	07	05	05	26	12	11	14	23	33	26	00
8	-0		36	99	24	18	29	40	76	20	66	87	78	76	27	50	03	08
8	4-		54	26	33	78	95	19	57	85	82	22	22	83	96	72	62	72
8	28																	
2	20	0.	0.	0.	0.	0.	0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.
4	22	41	02	08	10	05	12	09	11	09	29	15	12	19	21	35	28	04
8	-0		95	18	12	35	51	73	17	56	01	63	79	08	03	74	52	44
8	4-		05	45	72	57	24	48	44	52	64	69	62	62	33	85	94	55
9	29																	
2	20	-2.	-0.	0.	0.	0.	0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.
4	22	61	02	03	05	00	11	08	09	05	29	12	06	13	13	33	24	00
8	-0		59	01	03	94	05	80	50	02	00	42	93	47	04	00	92	80
9	5-		06	22	66	04	28	34	37	30	25	92	27	76	92	29	82	54
0	02																	
2	20	2.	0.	0.	0.	0.	0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.	-0.
4	22	04	02	07	11	04	12	08	08	04	27	13	10	15	17	33	26	01
8	-0		55	11	18	18	03	19	40	50	23	39	93	77	29	48	42	09
9	5-		52	67	19	73	00	55	83	39	33	64	07	15	54	60	72	70
1	03																	

```
In [8]: Xdf, ydf = df.iloc[:, 2:-1], df.iloc[:, -1]
X = Xdf.astype("float32")
y = ydf.astype("float32")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=ts, shuffle=False
) # It is important to keep "shuffle=False"
n_features = X_train.shape[1]
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

(14936, 15) (9956, 15) (14936,) (9956,)

```
In [9]: # Scaling

from sklearn.preprocessing import MinMaxScaler

scaler_input = MinMaxScaler(feature_range=(-1, 1))
scaler_input.fit(X_train)
X_train = scaler_input.transform(X_train)
X_test = scaler_input.transform(X_test)

mean_ret = np.mean(y_train) # Useful to compute the performance = R2

scaler_output = MinMaxScaler(feature_range=(-1, 1))
y_train = y_train.values.reshape(len(y_train), 1)
y_test = y_test.values.reshape(len(y_test), 1)
scaler_output.fit(y_train)
y_train = scaler_output.transform(y_train)
y_test = scaler_output.transform(y_test)
```

2.2. MLP Model and Training

Now that we have all our data ready, let's build our first MLP model. To that end, we will define several things:

- **Activation function**

Although the different hidden layers may employ different activation functions, in this case, we will select the same one for all hidden layers: the rectified linear unit (**ReLU**).

- **Hidden layers and units within layers**

In this model, we will use a total of 3 hidden layers. Each of these layers will have 50, 30, and 10 units respectively in order from the input layer.

- **Output layer**

Of course, we need to define a final fully connected layer for the output

- **Learning rate**

As before, we will choose a learning rate of 10^{-5} . (Note that we will play around with this hyperparameter--i.e., tuning it--in the forthcoming Deep Learning in Finance course.)

- **Optimizer**

We will choose the **Adam** optimizer, which you are already familiar with.

- **Loss function**

Different from the linear regression case, here we will select a loss function based on the **mean absolute error (MAE)**. This is:

$$L(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Note that, importantly, this error function has a different treatment of outliers than the MSE (mean squared error) function that we have used in linear regression (essentially because it does not compute the power of the differences!). This can be important, especially in the finance context, when we do not want to give too much importance to a few outliers that are simply "extreme events," not representative of the overall sample. Of course, it would be a completely different story if our model aimed at predicting extreme events (e.g., firm bankruptcy, client default on credit card debt, etc.), since these extreme values will be the main focus of our investigation.

****NOTE, also, that we are setting our seed to be constant**. This is important to note, because different training could lead to different final outcomes due to a variety of things (e.g., random weight initialization in Keras). You are welcome to remove this seed and check how things change!*

```
In [10]: import tensorflow as tf

tf.random.set_seed(12345)

act_fun = "relu" # Activation function
hp_units = 50 # Units in the first hidden layer
hp_units_2 = 30 # Units in the second hidden layer
hp_units_3 = 10 # Units in the third hidden layer

model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Dense(units=hp_units, activation=act_fun))
model.add(tf.keras.layers.Dense(units=hp_units_2, activation=act_fun))
model.add(tf.keras.layers.Dense(units=hp_units_3, activation=act_fun))
model.add(tf.keras.layers.Dense(1))

hp_lr = 1e-5

adam = tf.keras.optimizers.Adam(learning_rate=hp_lr)
model.compile(optimizer=adam, loss="mean_absolute_error")
```

```
2026-08-29 05:17:14.904907: I tensorflow/core/util/port.cc:153] oneDNN
custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-08-29 05:17:15.751693: I
tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is
optimized to use available CPU instructions in performance-critical
operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16
AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
2026-08-29 05:17:19.061383: I tensorflow/core/util/port.cc:153] oneDNN
custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

Once we have defined our model, we can train it!

```
In [11]: model.fit(X_train, y_train, epochs=30, batch_size=32, verbose=2)
```

Epoch 1/30
467/467 - 3s - 6ms/step - loss: 0.1726
Epoch 2/30
467/467 - 1s - 2ms/step - loss: 0.1391
Epoch 3/30
467/467 - 1s - 2ms/step - loss: 0.1297
Epoch 4/30
467/467 - 1s - 3ms/step - loss: 0.1275
Epoch 5/30
467/467 - 1s - 2ms/step - loss: 0.1264
Epoch 6/30
467/467 - 1s - 3ms/step - loss: 0.1256
Epoch 7/30
467/467 - 1s - 3ms/step - loss: 0.1250
Epoch 8/30
467/467 - 1s - 3ms/step - loss: 0.1245
Epoch 9/30
467/467 - 1s - 3ms/step - loss: 0.1240
Epoch 10/30
467/467 - 1s - 3ms/step - loss: 0.1235
Epoch 11/30
467/467 - 1s - 3ms/step - loss: 0.1231
Epoch 12/30
467/467 - 1s - 3ms/step - loss: 0.1227
Epoch 13/30
467/467 - 1s - 3ms/step - loss: 0.1223
Epoch 14/30
467/467 - 1s - 2ms/step - loss: 0.1219
Epoch 15/30
467/467 - 1s - 3ms/step - loss: 0.1216
Epoch 16/30
467/467 - 1s - 2ms/step - loss: 0.1213
Epoch 17/30
467/467 - 1s - 3ms/step - loss: 0.1210
Epoch 18/30
467/467 - 1s - 3ms/step - loss: 0.1208
Epoch 19/30
467/467 - 1s - 3ms/step - loss: 0.1205
Epoch 20/30
467/467 - 1s - 3ms/step - loss: 0.1203
Epoch 21/30
467/467 - 1s - 3ms/step - loss: 0.1201
Epoch 22/30
467/467 - 1s - 2ms/step - loss: 0.1198
Epoch 23/30
467/467 - 1s - 3ms/step - loss: 0.1196
Epoch 24/30
467/467 - 1s - 2ms/step - loss: 0.1194
Epoch 25/30
467/467 - 1s - 2ms/step - loss: 0.1192
Epoch 26/30
467/467 - 1s - 3ms/step - loss: 0.1190
Epoch 27/30
467/467 - 1s - 3ms/step - loss: 0.1188
Epoch 28/30
467/467 - 1s - 2ms/step - loss: 0.1186
Epoch 29/30
467/467 - 1s - 3ms/step - loss: 0.1184
Epoch 30/30
467/467 - 1s - 2ms/step - loss: 0.1182

```
Out[11]: <keras.src.callbacks.history.History at 0x70694a31e850>
```

As before, we can get a feeling of how the model works by calling 'model.summary()':

```
In [12]: model.summary()
```

Model: "sequential"

Layer (type) Param #	Output Shape	
dense (Dense) 800	(None, 50)	
dense_1 (Dense) 1,530	(None, 30)	
dense_2 (Dense) 310	(None, 10)	
dense_3 (Dense) 11	(None, 1)	

Total params: 7,955 (31.08 KB)

Trainable params: 2,651 (10.36 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 5,304 (20.72 KB)

Now that we have a few hidden layers with several units in each one, being able to interpret the architecture and understand how many parameters are trained at each step of the model becomes crucial.

In this case, the 'summary()' option of Keras is already telling us how many parameters are trained in each layer. But where do these parameters come from?

For example, in the first layer of the model there are 800 parameters. These are equal to the number of units in the layer (50) times the number of different inputs (15)-because there would be a weight associated with each input and unit in the layer-plus the bias terms b for each unit (50). Thus, $15 \times 50 + 50 = 800$.

Where does the number of 1530 parameters from the second layer come from? This is equal to the number of "inputs" to this layer (which is essentially the number of units in the previous layer (50) times the number of units in the layer (30) plus the bias term for each unit (30). Thus, $50 \times 30 + 30 = 1530$.

You can check the number of parameters in the other layers using the same logic.

2.3. Validation and Early stopping

- **How many epochs should we use when training our models?**

These are very legitimate questions to ask at this point. The choice of the number of epochs that we have used so far (30) is completely discretionary and not based at all on any relevant criteria. We are going to change this now by including **Early stopping** in our training.

This means that we are going to instruct Keras to stop model training when some condition is met. This will be done via the **callback API** in Keras, as we will see shortly.

- **When should we stop training?**

There are multiple ways to define a stopping criterion. We are going to use one of the most common and focus on monitoring the loss function in a separate validation set. Once after each epoch of the training process, we will check if (and how much) the loss function in the validation set decreases. We will also define a parameter, **patience**, that indicates the number of epochs with no improvement in the validation set that we tolerate before Early stopping training.

*More information on how **Callback API** and **Early stopping** in Keras works can be found here: https://keras.io/api/callbacks/early_stopping/*

Let's therefore implement Early stopping when training our algorithm. First, we need to define the characteristics of Early stopping, where we indicate:

1. **The quantity/set to monitor:** in our case the validation set loss function (we will define the validation set in a minute).
2. **The 'mode':** by setting this to 'min' we ensure training will stop when the quantity set in (1) has stop decreasing.
3. **Patience:** we will allow for 10 epochs with no improvement in minimizing the loss function of the validation set before we stop the training.
4. **restore_best_weights:** due to the iteration process, it may be the case that the last iteration before stopping training does not yield the model weights that achieve the lowest loss function in validation. By setting this option to 'True' we ensure that we keep the weights that achieved the best loss function value (i.e., the lowest) in validation.

```
In [13]: es = tf.keras.callbacks.EarlyStopping(
          monitor="val_loss", mode="min", verbose=1, patience=10,
          restore_best_weights=True
        )
```

Finally, we can re-train our model with the Early stopping callback. Also, note that, because we need it for the Early stopping function, we define a validation set of 20% of the training set. Importantly, this split is performed by Keras with no shuffling of the data, and this set is kept apart (i.e., there is no training nor testing going on in the validation set). Lastly, note that we incorporate the callback feature 'es' defined before.

We have selected here 100 epochs for training but included an Early stopping criterion. Would the model train for the whole 100 epochs? Let's see...All rights reserved WQU WorldQuant University QQQQ

```
In [14]: model.fit(
          X_train,
          y_train,
          validation_split=0.2,
          epochs=100,
          batch_size=32,
          verbose=2,
          callbacks=[es],
        )
```

```
Epoch 1/100
374/374 - 1s - 3ms/step - loss: 0.1169 - val_loss: 0.1249
Epoch 2/100
374/374 - 1s - 3ms/step - loss: 0.1166 - val_loss: 0.1256
Epoch 3/100
374/374 - 1s - 3ms/step - loss: 0.1164 - val_loss: 0.1259
Epoch 4/100
374/374 - 1s - 3ms/step - loss: 0.1162 - val_loss: 0.1262
Epoch 5/100
374/374 - 1s - 4ms/step - loss: 0.1161 - val_loss: 0.1265
Epoch 6/100
374/374 - 1s - 3ms/step - loss: 0.1159 - val_loss: 0.1267
Epoch 7/100
374/374 - 1s - 3ms/step - loss: 0.1157 - val_loss: 0.1270
Epoch 8/100
374/374 - 1s - 4ms/step - loss: 0.1156 - val_loss: 0.1271
Epoch 9/100
374/374 - 1s - 4ms/step - loss: 0.1154 - val_loss: 0.1272
Epoch 10/100
374/374 - 1s - 3ms/step - loss: 0.1153 - val_loss: 0.1275
Epoch 11/100
374/374 - 1s - 2ms/step - loss: 0.1151 - val_loss: 0.1276
Epoch 11: early stopping
Restoring model weights from the end of the best epoch: 1.
```

```
Out[14]: <keras.src.callbacks.history.History at 0x706933f7a790>
```

As you can see, given our new criteria, model training has stopped at epoch 11, when the Early stopping kicks in.

Now, let's see if our simple deep learning model is able to produce a better prediction of the momentum factor returns than our linear regression did!

3. MLP Model Performance for Momentum Timing

As we did in the case of linear regression, we will::

1. Evaluate the predictive performance of our MLP-based: Out-of-sample R^2
2. Assess the viability of this by backtesting our trading strategy

Let's start with the first one.

3.1. Out-of-Sample Predictive Power (R^2_{OS})

Let's first look at the out-of-sample explanatory power of our model via the R^2_{OS} in Campbell and Thompson (2008). The construction of this measure is identical to what we did in Lesson 2 of the module. Please refer there if there are doubts in this regard.

```
In [15]: values = scaler_output.inverse_transform(y_test)

y_pred = model.predict(X_test)
y_pred = scaler_output.inverse_transform(y_pred)
```

312/312 ————— 0s 1ms/step

```
In [16]: y_pred.shape
```

```
Out[16]: (9956, 1)
```

```
In [17]: def R2_campbell(y_true, y_predicted, mean_ret):
          y_predicted = y_predicted.reshape((-1,))
          sse = sum((y_true - y_predicted) ** 2)
          tse = sum((y_true - mean_ret) ** 2)
          r2_score = 1 - (sse / tse)
          return r2_score

R2_Campbell = R2_campbell(values.flatten(), y_pred.flatten(), mean_ret)

print("R2 (Campbell): ", R2_Campbell)
```

```
R2 (Campbell): 0.021339238
```

Now, our R^2_{OS} is actually a big improvement over the linear regression case. Indeed, these kind of numbers are in the neighborhood of what is published in leading finance journals in terms of return predictability.

Note that there is some random weight initialization (among other things) going on when training the model, so if you run this model without setting the seed you will get different numbers. This will also impact the trading strategy returns. Feel free to try it!

Anyway, predictability is not a super-high number, so let's see if we can make actual money (over the buy-and-hold case) by following a trading strategy based on our models' predictions.

3.2. Backtesting Momentum Timing

As before, this section is inherently identical in construction to the one from Lesson 2, with the only changes coming from the predictions delivered by our MLP-based model. If you have doubts on how to perform some of the steps in this section, please go back to Lesson 2 of the module.

- **What do predicted versus real returns in a test set look like?**

```
In [18]: df_predictions = pd.DataFrame(  
    {  
        "Date": test_time.flatten(),  
        "Pred": y_pred.flatten(),  
        "Ret": (Ret_vector.flatten() / 100),  
        "Values": values.flatten(),  
    }  
)  
df_predictions.tail()
```

```
Out[18]:
```

	Date	Pred	Ret	Values
9951	2022-04-27	-0.006673	0.0194	-0.041617
9952	2022-04-28	-0.003995	-0.0074	-0.000872
9953	2022-04-29	-0.002553	0.0041	-0.044455
9954	2022-05-02	0.015910	-0.0261	-0.008054
9955	2022-05-03	0.000927	0.0204	-0.010970

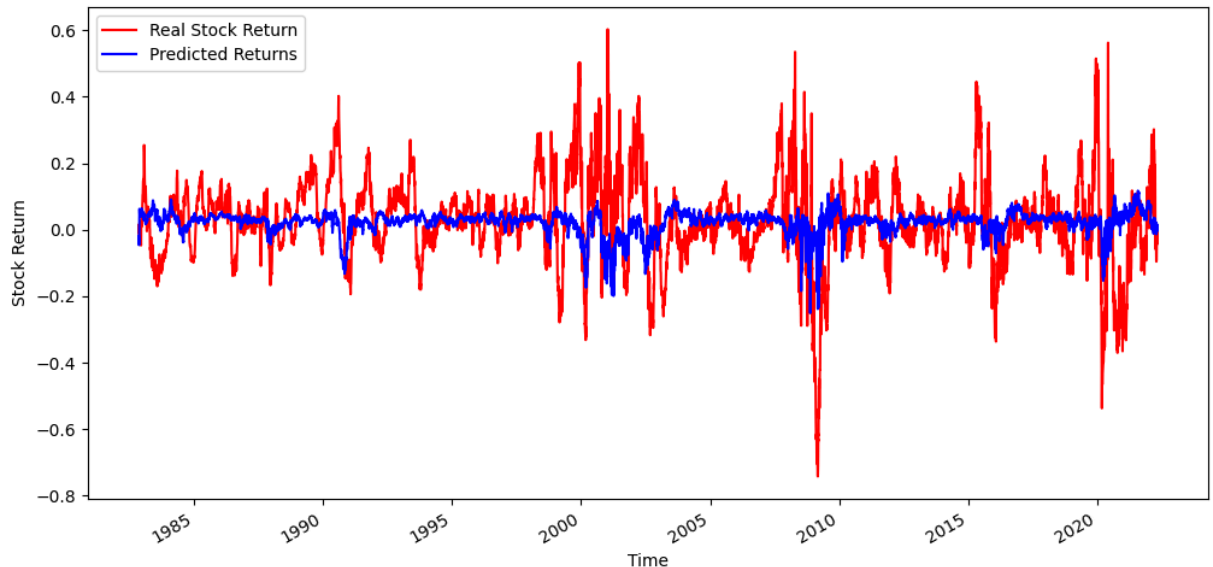
```
In [19]: df_predictions.Date = pd.to_datetime(df_predictions.Date, format="%YYYY-  
%mm-%dd")  
df = df_predictions  
df.tail()
```

```
Out[19]:
```

	Date	Pred	Ret	Values
9951	2022-04-27	-0.006673	0.0194	-0.041617
9952	2022-04-28	-0.003995	-0.0074	-0.000872
9953	2022-04-29	-0.002553	0.0041	-0.044455
9954	2022-05-02	0.015910	-0.0261	-0.008054
9955	2022-05-03	0.000927	0.0204	-0.010970

```
In [20]: import matplotlib.pyplot as plt
```

```
fig = plt.figure(figsize=(12, 6))
ax = plt.gca()
df.plot(x="Date", y="Values", color="red", label="Real Stock Return",
ax=ax)
df.plot(x="Date", y="Pred", color="blue", label="Predicted Returns",
ax=ax)
plt.xlabel("Time")
plt.ylabel("Stock Return")
plt.legend()
plt.show()
```



As you can see, the predicted returns are much less volatile (noisy) in the case of the MLP model than in the linear regression case. This has to do with the enhanced predictive power of MLPs. Let's now see whether this predictive power is enough to effectively time momentum returns.

- **Momentum timing strategy**

```
In [21]: df["Positions"] = df["Pred"].apply(np.sign)
df["Strat_ret"] = df["Positions"].shift(1) * df["Ret"]
df["Positions_L"] = df["Positions"].shift(1)
df["Positions_L"][df["Positions_L"] == -1] = 0
df["Strat_ret_L"] = df["Positions_L"] * df["Ret"]
df["CumRet"] = df["Strat_ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)
df["CumRet_L"] = df["Strat_ret_L"].expanding().apply(lambda x: np.prod(1 + x) - 1)
df["bhRet"] = df["Ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)

Final_Return_L = np.prod(1 + df["Strat_ret_L"]) - 1
Final_Return = np.prod(1 + df["Strat_ret"]) - 1
Buy_Return = np.prod(1 + df["Ret"]) - 1

print("Strat Return Long Only =", Final_Return_L * 100, "%")
print("Strat Return =", Final_Return * 100, "%")
print("Buy and Hold Return =", Buy_Return * 100, "%")
```

```
/tmp/ipykernel_18/4012059038.py:4: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in
certain cases, but when using Copy-on-Write (which will become the default
behaviour in pandas 3.0) this will never work to update the original
DataFrame or Series, because the intermediate object on which we are setting
values will behave as a copy.
A typical example is when you are setting values in a column of a DataFrame,
like:
```

```
df["col"][row_indexer] = value
```

```
Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment
in a single step and ensure this keeps updating the original `df`.
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
```

```
df["Positions_L"][df["Positions_L"] == -1] = 0
/tmp/ipykernel_18/4012059038.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
```

```
df["Positions_L"][df["Positions_L"] == -1] = 0
```

```
Strat Return Long Only = 6172.663616358873 %
```

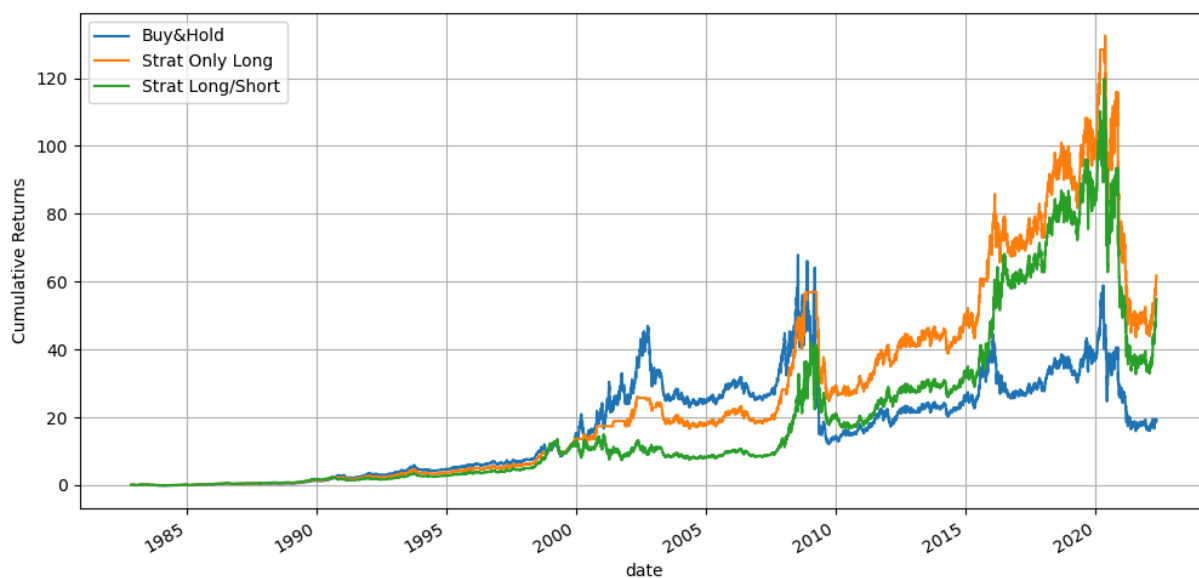
```
Strat Return = 5480.8697405290795 %
```

```
Buy and Hold Return = 1909.8411102470113 %
```

```
In [22]: import matplotlib.pyplot as plt
```

```
fig = plt.figure(figsize=(12, 6))
ax = plt.gca()
df.plot(x="Date", y="bhRet", label="Buy&Hold", ax=ax)
df.plot(x="Date", y="CumRet_L", label="Strat Only Long", ax=ax)
df.plot(x="Date", y="CumRet", label="Strat Long/Short", ax=ax)
plt.xlabel("date")
plt.ylabel("Cumulative Returns")
plt.grid()
plt.show()

df.describe()
```



Out[22]:

	Date	Pred	Ret	Values	Positions	Strat_ret	Positions_L	Strat_ret_L	Cum Ret	Cum Ret_L	bhRet
count	9956	9956 .000 000	9956 .000 000	9956 .000 000	9956 .000 000	9955 .000 000	9955 .000 000	9955 .000 000	9955 .000 000	9955 .000 000	9956 .000 000
mean	2002-07-28 08:01:20.996384128	0.022021	0.000437	0.027431	0.712334	0.000537	0.856153	0.000487	19.087950	26.345214	16.782598
min	1982-11-05 00:00:00	-0.250808	-0.205800	-0.742601	-1.000000	-0.205800	0.000000	-0.205800	-0.158656	-0.178537	-0.203779
25%	1992-09-09 18:00:00	0.015296	-0.005700	-0.032436	1.000000	-0.005900	1.000000	-0.004000	1.986957	2.532985	3.261404
50%	2002-07-25 12:00:00	0.029147	0.001000	0.028531	1.000000	0.000800	1.000000	0.000000	9.162204	18.238026	16.947453
75%	2012-06-12 06:00:00	0.038782	0.007000	0.091134	1.000000	0.006800	1.000000	0.005200	27.970613	42.367808	26.658081
max	2022-05-03 00:00:00	0.117392	0.143300	0.603302	1.000000	0.170900	1.000000	0.124300	119.843680	132.542472	67.883048
std	NaN	0.033852	0.016345	0.128949	0.701876	0.016343	0.350952	0.011923	24.186603	29.212679	13.714998

Sadly, as you can see, our strategy is not really able to time momentum factor returns. Arguably, our MLPs do a much better job than the linear regression case, but we still need much to be able to outperform the buy-and-hold strategy. This is not really surprising, since momentum has been one of the most profitable factors ever!

4. Conclusion

Hopefully, you are not too disappointed by the failure of our trading strategies. Some of you may already be familiar with deep learning applications in other

fields, where DL models achieve a remarkable performance and predictive power. Unfortunately, predicting the stock market is not as easy a task for an algorithm as recognizing and classifying pictures into groups of cats and dogs.

Luckily for us, there is still a lot that can be done to improve the predictive performance of our algorithms in a financial market setting: hyperparameter tuning, more dense and complex networks, feeding data from other different sources, etc. We will deal with all these issues (and more) in the upcoming Deep Learning in Finance course.

See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.